

Secret Status: Top Secret () Secret () Restrict (☒) Public ()

RKNanoD GUI User Manual (Version 1.0)

Product R&D Dept.III
Rockchip Electronics Co.,Ltd
www.rock-chips.com

Status: ☐ development ☒ public ☐ modify	File Tag:	GUI User Manual	
	Version:	1.0	
	Author:	Benjo.lei	
	Data:	2016 年 4 月 10 日	
	Verify:	Zheng Yongzhi	2016 年 4 月 15 日

Version History

Version	Author	Date	Remarks
1.0	Benjo.lei	2016-04-10	Original

目录

1	GUI 概要	11
1.1	GUI 概述.....	11
1.2	GUI 架构关系图.....	11
1.3	GUI 架构说明.....	12
1.4	GUI 目录说明.....	12
2	UI 管理器	13
2.1	UI 管理器简介.....	13
2.2	UI 管理器架构说明.....	13
2.3	GUITask.....	14
2.4	UI 定时器	16
2.4.1	rkgui_create_timer	16
2.4.2	rkgui_delete_timer	16
2.4.3	rkgui_start_timer	17
2.4.4	rkgui_stop_timer	17
2.5	遮叠管理器.....	17
2.5.1	遮叠管理流程.....	18
2.5.2	GcRelevanceRegister	19
2.5.3	GcRelevanceUnRegister	19
2.5.4	GcRelevanceOper	19
2.5.5	GcDisplay.....	20
2.5.6	GcUpdataRegister	20
2.5.7	TraversalUpdataList	20
2.6	控件管理器.....	20
2.6.1	GUI_GcCreate.....	21
2.6.2	GUI_GcDelete.....	21
2.6.3	GUI_GcDelete.....	22
2.6.4	GUI_OperWidge	22
2.6.5	GcDisplay.....	22

2.7	消息与焦点管理器	23
2.7.1	GuiTask_AppReciveMsg	23
2.7.2	GuiTask_AppUnReciveMsg	23
2.7.3	GuiTask_Setfocus.....	23
2.7.4	GuiTask_Delfocus.....	24
2.8	GUI API 介绍	24
2.8.1	GuiTask_CreateWidget	24
2.8.2	GuiTask_DeleteWidget	24
2.8.3	GuiTask_OperWidget.....	25
2.8.4	GuiTask_ScreenLock	25
2.8.5	GuiTask_ScreenUnLock	25
2.8.6	GuiTask_Setfocus.....	26
2.8.7	GuiTask_Delfocus.....	26
2.8.8	GuiTask_AppReciveMsg	26
2.8.9	GuiTask_UnAppReciveMsg	27
3	ICON 控件	27
3.1	ICON 控件简介	27
3.2	ICON 创建	27
3.2.1	RKGUI_ICON_ARG	27
3.2.2	参考例程.....	28
3.3	ICON 内容变更	28
3.3.1	变更图标.....	28
3.4	ICON 创建流程	29
3.5	实现说明	29
3.5.1	GUI_ICON_CLASS	29
3.5.2	GUI_IconCreate	30
3.5.3	GUI_IconDelete	30
3.5.4	GUI_IconSetID	31
3.5.5	GUI_IconInit.....	31

3.5.6	GUI_IconDisplay	31
3.5.7	GUI_IconDisplayResource	31
3.5.8	GUI_IconAreaUpdata	32
4	IMAGE 控件.....	33
4.1	IMAGE 控件简介.....	33
4.2	IMAGE 创建.....	33
4.2.1	RKGUI_IMAGE_ARG.....	33
4.2.2	矩形框模式.....	34
4.3	IMAGE 创建流程.....	35
4.4	实现说明.....	36
4.4.1	GUI_IMAGE_CLASS	36
4.4.2	GUI_ImageCreate	37
4.4.3	GUI_ImageDelete	37
4.4.4	GUI_ImageDisplay	37
4.4.5	GUI_ImageDisplayRect	38
5	TEXT 控件	38
5.1	TEXT 控件简介.....	38
5.2	TEXT 创建	38
5.2.1	RKGUI_TEXT_ARG	39
5.2.2	普通模式.....	40
5.2.3	SID 模式	41
5.3	TEXT 内容变更.....	42
5.3.1	变更文字.....	42
5.3.2	变更字体颜色.....	42
5.3.3	变更背景图片/背景色.....	42
5.3.4	变更对齐方式.....	43
5.4	TEXT 创建流程.....	43
5.5	实现说明.....	45
5.5.1	GUI_TEXT_CLASS	45

5.5.2	GUI_TextCreate	47
5.5.3	GUI_TextDelete	47
5.5.4	GUI_TextDisplay	48
5.5.5	GUI_TextSetContent.....	48
5.5.6	GUI_TextSetContent.....	48
5.5.7	GUI_TextDisplayID	49
5.5.8	GUI_TextDisplayBuff.....	50
5.5.9	GUI_BmpFronDisplay	50
5.5.10	GUI_TextSetMove	51
5.5.11	GUI_TextTimer	51
5.5.12	GUI_TextExtractString	52
5.5.13	GUI_TextExtractString	52
5.5.14	GUI_TextSizeCmp	53
5.5.15	GUI_TextExtractSize	53
5.5.16	Font16_CompData / Font12_CompData.....	54
5.5.17	Font12_CompType / Font16_CompType.....	54
6	MSGBOX 控件.....	54
6.1	MSGBOX 控件简介	54
6.2	MSGBOX 创建.....	55
6.2.1	RKGUI_MSGBOX_ARG.....	55
6.2.2	警告模式.....	56
6.2.3	对话框模式.....	57
6.3	MSGBOX 回调处理.....	57
6.3.1	警告模式回调	58
6.3.2	对话框模式回调	58
6.4	MSGBOX 创建流程.....	60
6.5	实现说明.....	62
6.5.1	GUI_MSGBOX_CLASS	62
6.5.2	GUI_MsgBoxCreate	63

6.5.3	GUI_MsgBoxDelete	63
6.5.4	GUI_MsgBoxDisplay.....	64
6.5.5	GUI_MsgBoxCallBack	64
7	PROCESSBAR 控件	64
7.1	PROCESSBAR 控件简介	64
7.2	PROCESSBAR 创建	65
7.2.1	RKGUI_PROGRESSBAR_ARG	65
7.2.2	参考例程.....	66
7.3	PROCESSBAR 内容变更	66
7.3.1	长度变更.....	66
7.4	PROCESSBAR 创建流程	67
7.5	实现说明.....	69
7.5.1	GUI_PROGRESSBAR_CLASS.....	69
7.5.2	GUI_ProgressBarCreate.....	69
7.5.3	GUI_ProgressBarDelete.....	70
7.5.4	GUI_ProgressBarDisplay.....	70
7.5.5	GUI_ProgressBarContent.....	70
8	CHAIN 控件.....	71
8.1	CHAIN 控件简介	71
8.2	CHAIN 创建	71
8.2.1	RKGUI_CHAIN_ARG	71
8.2.2	示例代码.....	72
8.3	CHAIN 创建流程	73
8.4	实现说明.....	75
8.4.1	GUI_CHAIN_CLASS.....	75
8.4.2	GUI_ChainCreate.....	75
8.4.3	GUI_ChainDelete.....	76
8.4.4	GUI_ChainDisplay.....	76
9	SELECT 控件	76

9.1	SELECT 控件简介	76
9.2	SELECT 创建	77
9.2.1	RKGUI_SELECT_ARG	77
9.2.2	示例代码	78
9.3	SELECT 回调处理	78
9.3.1	SID 模式	79
9.3.2	普通模式	80
9.4	SELECT 内容变更	80
9.5	SELECT 创建流程	81
9.6	实现说明	82
9.6.1	GUI_SELECT_CLASS	82
9.6.2	GUI_SelectCreate	84
9.6.3	GUI_SelectDelete	84
9.6.4	GUI_SelectDisplay	84
9.6.5	DisplayMenuStrWithIDNum	84
9.6.6	GUI_SelectSetContent	85
9.6.7	GUI_SelectCallBack	85
9.6.8	GUI_SelectCallBackApp	85
9.6.9	GUI_SelectClearDrag	86
9.6.10	GUI_SelectDrag	86
9.6.11	GUI_CellCursorDisplay	86
9.6.12	GUI_ClearBackCursor	87
9.6.13	GUI_ItemDisplay	87
9.6.14	GUI_SelectStopTimer	88
9.6.15	GUI_SelectStartTimer	88
9.6.16	GUI_SelectStartTimer	88
10	SPECTRUM 控件	89
10.1	SPECTRUM 控件简介	89
10.2	SPECTRUM 创建	89

10.2.1	RKGUI_SPECTRUM_ARG.....	89
10.2.2	示例代码.....	90
10.3	内容变更.....	90
10.3.1	变更高度.....	91
10.4	SPECTRUM 创建流程.....	91
10.5	实现说明.....	92
10.5.1	GUI_SPECTRUM_CLASS	92
10.5.2	GUI_SelectCreate	92
10.5.3	GUI_SpectrumDelete.....	93
10.5.4	GUI_SpectrumDisplay	93
10.5.5	GUI_SpectrumContent.....	93
11	GUI 新增控件.....	94
11.1	GUI 控件概述.....	94
11.2	控件类型声明.....	94
11.3	控件类与控件子类.....	94
11.3.1	GUI_CONTROL_CLASS	94
11.3.2	示例代码.....	96
11.4	创建与删除接口	96
11.4.1	创建与删除接口	96
11.4.2	初始化/反初始化规则.....	97
11.5	控件操作接口	98
11.6	控件显示接口	99
12	硬件移植相关.....	99
12.1	移植概述.....	99
12.2	硬件抽象层架构.....	100
12.3	接口说明.....	100
12.3.1	LCD 设备参数.....	100
12.3.2	硬件参数类.....	101
12.3.3	接口类.....	101

12.3.4	LcdInit.....	102
12.3.5	LcdWakeUp.....	102
12.3.6	LcdStandby	102
12.3.7	LcdClear.....	103
12.3.8	LcdSendData.....	103
12.3.9	Lcd_Write.....	103
12.3.10	LcdSetWindow	104
12.3.11	LcdImage.....	104
12.3.12	LcdScreenUpdateDma	104
12.3.13	LcdSetEdian	105
12.3.14	LcdSetRotate	105
12.4	步骤简述.....	106
13	GUI 资源更新流程.....	106
13.1	工具简介.....	106
13.2	图片资源更新流程.....	106
13.3	菜单资源更新流程.....	107

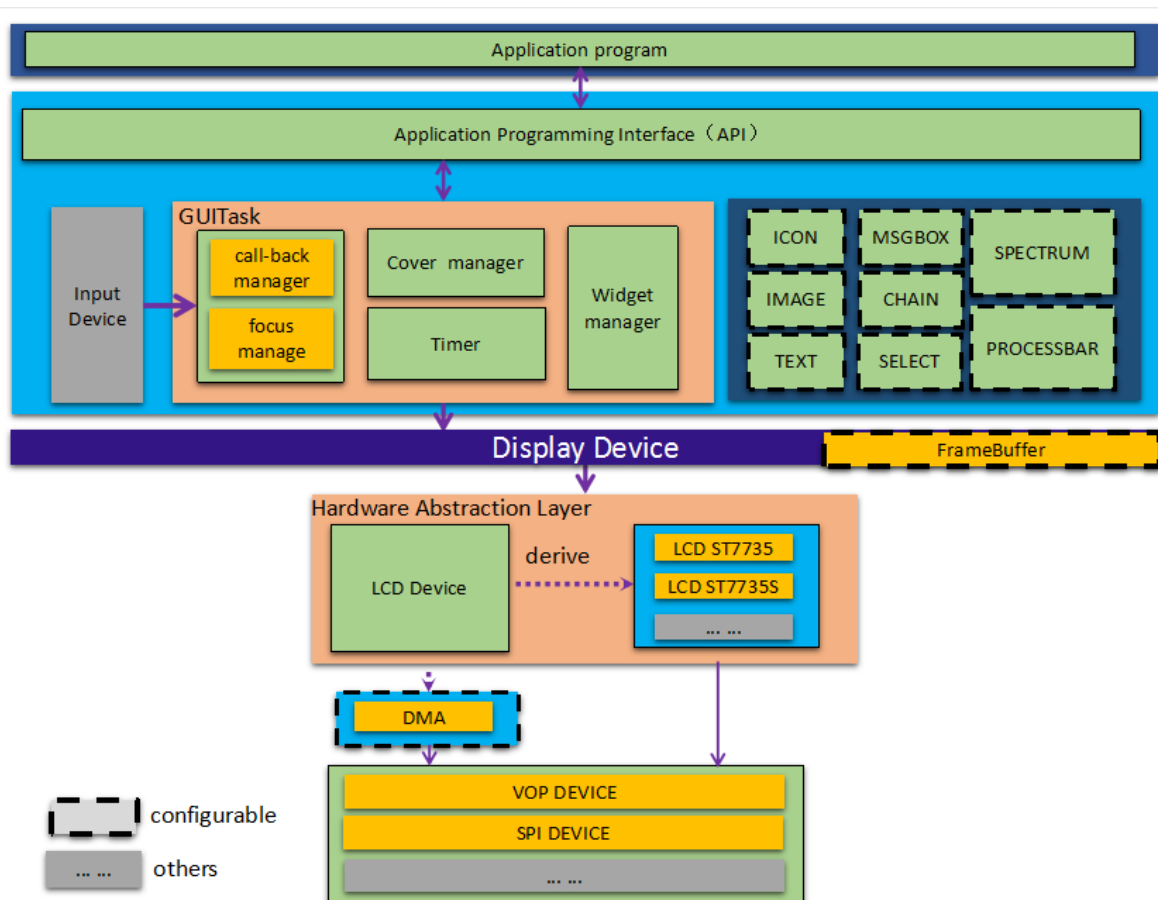
1 GUI 概要

1.1 GUI 概述

图形用户界面（Graphical User Interface，简称 GUI，又称图形用户接口）是一种人与计算机通信的界面显示格式，允许用户使用矩阵按键等输入设备操纵屏幕上的图标或菜单选项，以调用文件、启动程序或执行其它一些任务。

RKOS GUI 是由瑞芯微电子股份有限公司研发的一款针对 RKNano 系列芯片的开源定制化的图形用户界面，RKOS GUI 设计用于为任何基于 RKOS 的应用程序提供图形显示提供支持，并适用于任意尺寸的 LCD 图形显示。具有控件种类丰富、占用资源少、执行高效、稳定性好、可配置性强等特点。

1.2 GUI 架构关系图



1.3 GUI 架构说明

RKOS GUI 从结构上大致可分为：应用程序编程接口，软件算法层，硬件抽象层，驱动层四部分。其中 API（Application Programming Interface，应用程序编程接口）是一些预先定义的函数，目的是提供应用程序与开发人员基于某软件或硬件得以访问一组例程的能力；软件算法层是一组硬件无关的软件算法集合，是整个 GUI 系统的核心；硬件抽象层位于系统内核与硬件驱动之间，目的是将硬件抽象化，屏蔽硬件接口细节；驱动层，是控制硬件设备工作的程序，根据硬件接口细节的不同，驱动程序往往各不相同。RKOS GUI 的设计是平台无关的，根据 RKOS GUI 框架设计的驱动程序，能够与系统无缝接合，从而获得各类尺寸 LCD 图形显示器的支持。

1.4 GUI 目录说明

GUI 相关代码位于\Gui 中，以下是各目录及文件功能说明：

文件名称	功能说明
GUITask.c	GUI 管理线程，UI 管理器的核心实现。由 UI 定时器、回调管理、焦点管理等部分组成。
GUITask.h	GUI 管理线程头文件。
GUIManager.c	UI 管理器，由控件管理、遮叠管理两部分组成。
GUIManager.h	UI 管理器头文件。
GUIIcon.c	ICON 控件实现。
GUIIcon.h	ICON 控件头文件。
GUIImage.c	IMAGE 控件实现。
GUIImage.h	IMAGE 控件头文件。
GUIText.c	TEXT 控件实现。
GUIText.h	TEXT 控件头文件。
GUIMsgBox.c	MSGBOX 控件实现。
GUIMsgBox.h	MSGBOX 控件头文件。
GUIChain.c	CHAIN 控件实现。
GUIChain.h	CHAIN 控件头文件。

GUIProgressBar.c	PROGRESSBAR 控件实现。
GUIProgressBar.h	PROGRESSBAR 控件头文件。
UISpectrum.c	SPECTRUM 控件实现。
UISpectrum.h	SPECTRUM 控件头文件。
GUISelect.c	SELECT 控件实现。
GUISelect.h	SELECT 控件头文件。
LCDDriver.c	硬件抽象层，LCD 虚拟设备实现。
LCDDriver.h	LCD 虚拟设备头文件。
ST7735SDriver.c	LCD ST7735S 驱动文件。
ST7735SDriver.h	LCD ST7735S 驱动头文件。

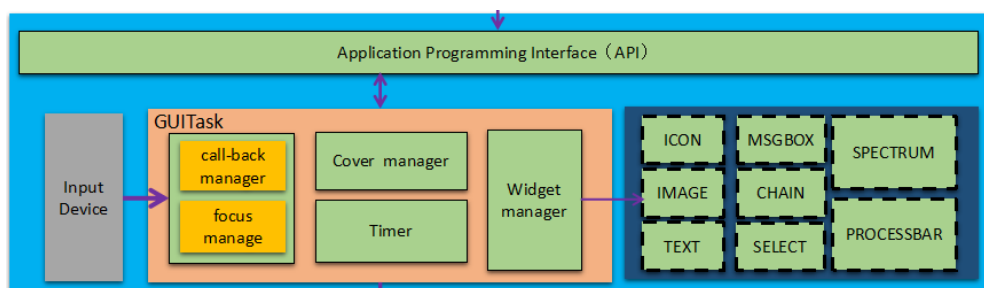
2 UI 管理器

2.1 UI 管理器简介

在 RKOS GUI 中，UI 管理器负责响应所有应用程序发来的事件请求，调用底层函数并返回正确的执行结果，是整个 GUI 架构的核心，也是和编写应用程序关系最为紧密的模块。

2.2 UI 管理器架构说明

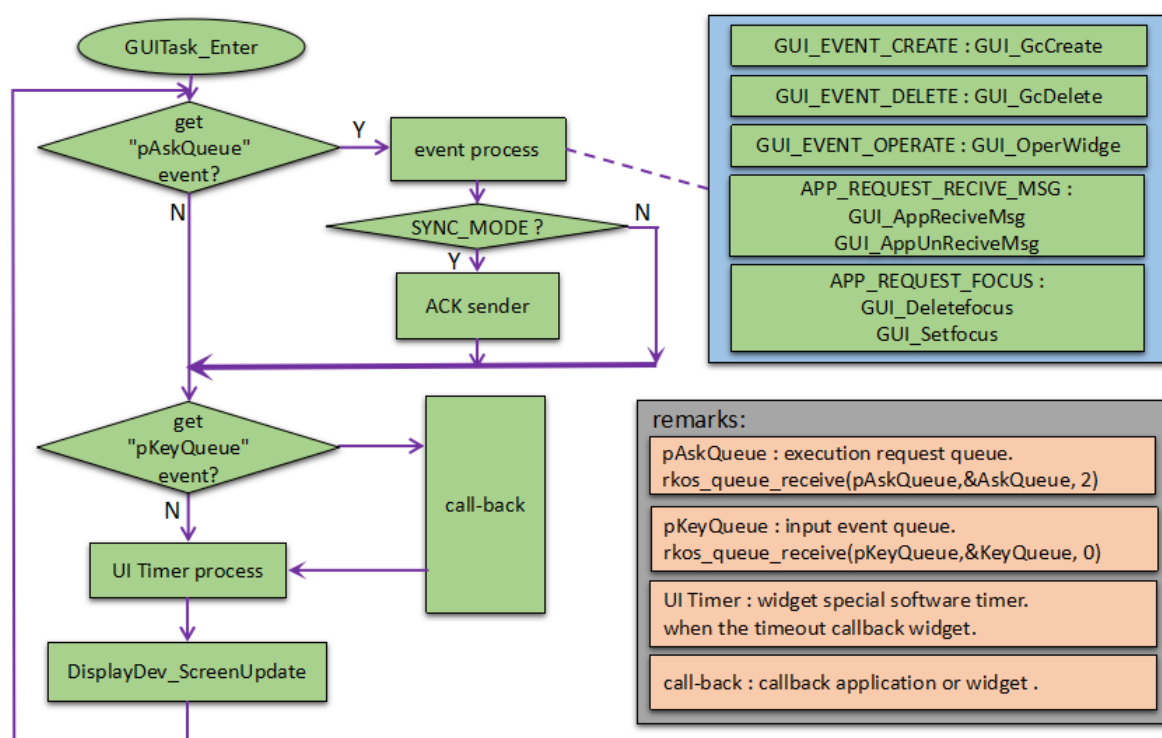
完整的 UI 管理器由 GUITask、UI 定时器、遮叠管理器、控件管理器、事件与焦点管理器、控件等部分组成。



2.3 GUITask

GUITask，即 GUI 线程。RKOS GUI 控件管理的本质是一个独立运行的线程，应用程序可以通过特定的 API 向 GUITask 发送命令，请求执行相应的动作。应用程序当可以选择等待 GUITask 完成这些动作后返回，即同步模式（SYNC_MODE）。也可以在事件发送完成后立即返回。即异步模式（ASYNC_MODE）。GUITask 的处理流程如下图所示：

GUITask 初始化完成后，将在 20ms 内不断地对事件请求队列进行监听。监听期间 GUITask 将释放至多 20ms 时间片，当消息队列中有未处理的消息时，GUITask 将立即取走响应。否则，在 20ms 延时后，GUITask 将会对输入事件队列进行检测，若队列中有未处理的消息，GUITask 将通过消息与焦点管理器通知应用程序或控件。当回调事件发生时，程序处于 GUITask 线程中，无法对消息队列进行操作。此时应用程序或控件再次以同步模式再次请求 GUITask 处理事件，则有可能造成线程锁死。因此，应用程序或控件不得在回调函数中使用 API，再次请求 GUI 服务。



```
while (1)
{
    if( rkos_queue_receive(pGUITaskControlBlock->pAskQueue,&AskQueue, 2) == RK_SUCCESS )
    {
        switch (AskQueue.event_type)
        {
            case GUI_EVENT_CREATE:
                GUI_CONTROL_CLASS * pGc;

                pGc= GUI_GcCreate(AskQueue.event,AskQueue.arg);
                RespQueue.arg= (void *)pGc;
                ret= RK_SUCCESS;
                break;

            case GUI_EVENT_DELETE:
                pGc= (GUI_CONTROL_CLASS *)AskQueue.arg;
                ret= GUI_GcDelete(pGc);
                break;

            case GUI_EVENT_OPERATE:
                {
                    RKGUI_OPERATE_ARG * pArg= (RKGUI_OPERATE_ARG *)AskQueue.arg;
                    ret= GUI_OperWidge(pArg->pGc, AskQueue.event, pArg->arg);
                }
                break;

            case APP_REQUEST_RECIVE_MSG:
                if ( AskQueue.event )
                    ret= GUI_AppUnRecvMsg(AskQueue.arg);
                else
                    ret= GUI_AppRecvMsg(AskQueue.arg);
                break;

            case APP_REQUEST_FOCUS:
                if ( AskQueue.event )
                    ret= GUI_Deletefocus(AskQueue.arg);
                else
                    ret= GUI_Setfocus(AskQueue.arg);
                break;

            case GUI_OPERATE_RESOURCE:
                ret= RK_SUCCESS;
                break;

            default :
                break;
        }

        RespQueue.error_code = ret;
        if (AskQueue.mode == SYNC_MODE)
        {
            rkos_queue_send(pGUITaskControlBlock->pRespQueue,&RespQueue, MAX_DELAY);
        }
    }
}

if( rkos_queue_receive(pGUITaskControlBlock->pKeyQueue,&KeyQueue, 0)== RK_SUCCESS )
{
    switch (KeyQueue.event_type)
    {
        case GUI_EVENT_KEY:
            {
                GUI_GC_RECIVE_MSG * pfGcFocusTemp = pGUITaskControlBlock->pfActiveFocus;

                if(pfGcFocusTemp != NULL)
                {
                    ret= pfGcFocusTemp->pfAppRecvMsg(KeyQueue.event_type ,KeyQueue.event, KeyQueue.arg, pfGcFocusTemp->pGc);
                    if(ret != RK_SUCCESS)
                    {
                        ret= GUI_CallbackApp(APP_RECIVE_MSG_EVENT_KEY, KeyQueue.event, KeyQueue.arg, NULL);
                    }
                    break;
                }

                ret= GUI_CallbackApp(APP_RECIVE_MSG_EVENT_KEY, KeyQueue.event, KeyQueue.arg, NULL);
                break;
            }

            default :
                break;
        }
    }
}
```

```
{
    GUI_CLASS_TIMER * pTimerTemp= pGUITaskControlBlock->pTimerList;

    while (pTimerTemp != NULL && ((pGUITaskControlBlock->TimerLock == 0) || ((GUI_CONTROL_CLASS *) (pTimerTemp->pGc))->level))
    {
        if (pTimerTemp->pGc!= NULL)
        {
            if(pTimerTemp->start)
            {
                pTimerTemp->pfTimerReciveMsg(pTimerTemp->pGc);
            }
        }
        pTimerTemp= pTimerTemp->next;
    }

    if(pGUITaskControlBlock->ScreenLock==0)
    {
        DisplayDev_ScreenUpdate( );
    }
}
```

2.4 UI 定时器

UI 定时器是一款专为 RKOS GUI 控件提供定时器服务的软件定时器，采用 GUI 唤醒次数作为时基，定时时间与 GUI 繁忙程度有关。控件可以通过 API 创建和使用 UI 定时器。

2.4.1 rkgui_create_timer

函数原型	rk_err_t rkgui_create_timer(HGC pGc, P_TEXT_RECVIE pfTimerReciveMsg)
pGc	控件句柄
pfTimerReciveMsg	回调函数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	用于创建定时器。

2.4.2 rkgui_delete_timer

函数原型	rk_err_t rkgui_delete_timer(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	删除定时器。

2.4.3 rkgui_start_timer

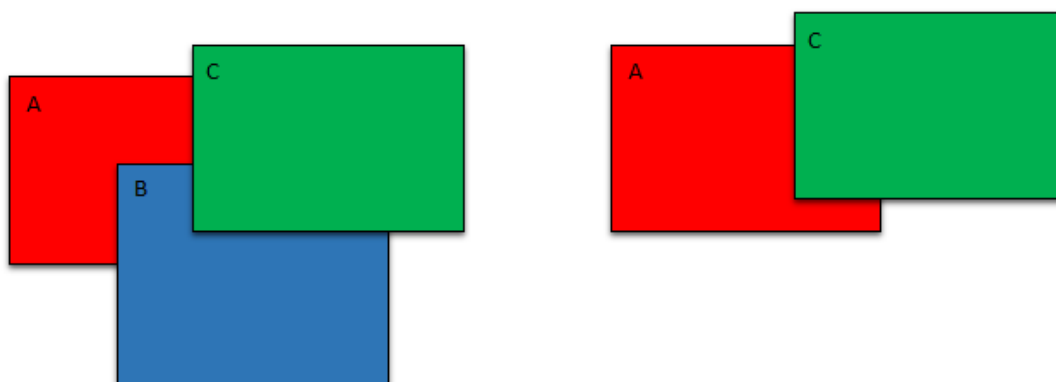
函数原型	rk_err_t rkgui_start_timer(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	开始计时。

2.4.4 rkgui_stop_timer

函数原型	rk_err_t rkgui_stop_timer(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	停止计时。

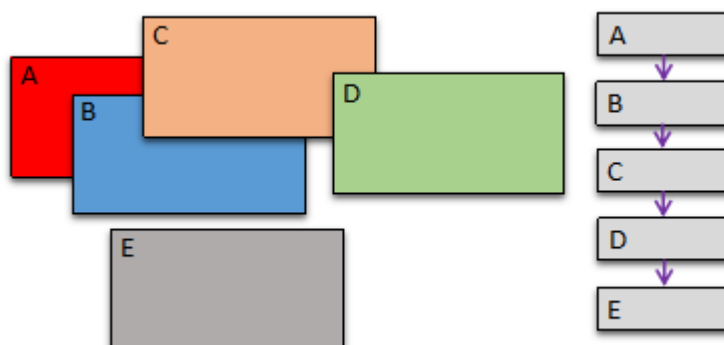
2.5 遮叠管理器

RKOS GUI 控件管理器中提供了一项自动控件遮叠管理的功能。以下图为例，当 A、B、C 三个控件互相交叠时，对任一控件的移动、隐藏等操作，将保持其他控件的遮叠关系。

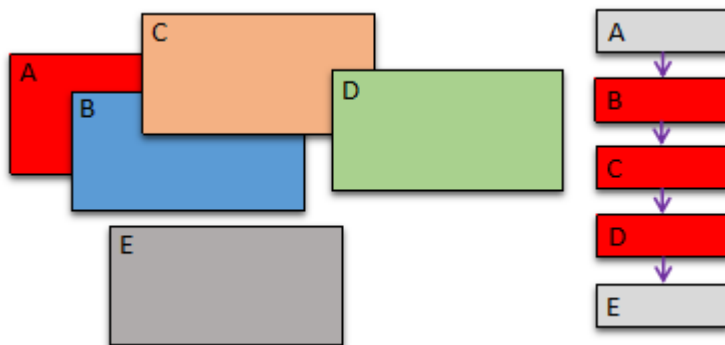


2.5.1 遮叠管理流程

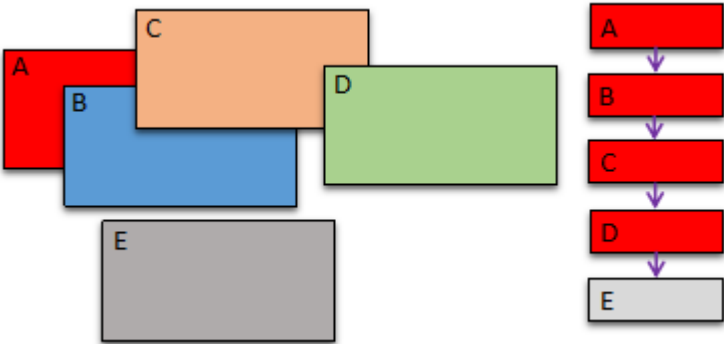
当 GUITask 接收到控件创建命令时，将调用 GcRelevanceRegister 函数，将控件加入链表进行管理。在实现如控件隐藏的功能时，控件可以通过调用 GcRelevanceOper 函数来对链表中所有需要重新绘制的控件进行标记。再通过 GcDisplay 函数来将被标记过的控件进行重绘。GcDisplay 函数将沿着控件链表（即根据控件的创建顺序）进行重绘。因此，先被创建的控件，始终位于下层。以下图为例，控件创建顺序依次为 A->B->C->D->E。



当需要判断控件 C 的交叠情况时，可以通过 GcUpdataRegister 函数，进行第一次交叠判断。GcUpdataRegister 函数将根据控件坐标信息，依次与整张链表中的所有元素进行交叠判断。并标记出所有与 C 交叠的控件。显然，可以得出 C 与 B、D 交叠的结论。



第一次交叠判断后，将生成如上图所示的参考表。然后可以通过 TraversalUpdataList 函数进行第二次交叠判断。TraversalUpdataList 函数将遍历参考表中所有控件，即根据第一次判断的规则，对控件 B 与控件 D 进行交叠判断。显然，能够得出 B 与 A、C 交叠，D 与 C 交叠。值得注意的是为了提升效率，所有的控件只会经过一次遍历。即第二次遍历时，控件 C 因为已经经过了搜索因此不会重复进行交叠判断。最后，调用 GcDisplay 函数，将沿控件链表将所有标记出的控件进行重绘。从而实现交叠管理功能。



2.5.2 GcRelevanceRegister

函数原型	rk_err_t GcRelevanceRegister(GUI_CONTROL_CLASS *pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	将控件加入链表。

2.5.3 GcRelevanceUnRegister

函数原型	rk_err_t GcRelevanceUnRegister(GUI_CONTROL_CLASS *pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	将控件移除链表。

2.5.4 GcRelevanceOper

函数原型	rk_err_t GcRelevanceOper(GUI_CONTROL_CLASS * pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	检索链表，并标记出所有带重绘的控件。

2.5.5 GcDisplay

函数原型	rk_err_t GcDisplay(GUI_CONTROL_CLASS * pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	重绘所有标记过的控件。

2.5.6 GcUpdateRegister

函数原型	rk_err_t GcUpdateRegister(GUI_CONTROL_CLASS * list, GUI_CONTROL_CLASS * pGc)
list	控件链表
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	标记出链表 list 中所有与控件 pGc 遮叠的控件，生成参考表。

2.5.7 TraversalUpdateList

函数原型	rk_err_t TraversalUpdateList(GUI_CONTROL_CLASS *list)
list	控件链表
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据参考表，标记出所有关联控件。

2.6 控件管理器

控件是对数据和方法的封装，在开发构建用户界面（UI）时，使用 RKOS GUI 所提供的控件，能够快速便捷地完成文字、图标、动画等界面元素的操作。对所有这些控件统一操作、管理的模块便称为控件管理器。当前版本中，RKOS GUI 提供了下列 8 种控件：

- ❖ ICON 控件：图标控件，用于显示固件中预存的图标。
- ❖ IMAGE 控件：影像控件，用于绘制纯色矩形框、矢量图或一组内存数据。
- ❖ TEXT 控件：文本控件，用于显示 unicode 文字。
- ❖ MSGBOX 控件：消息控件，用于弹出警告框或对话框。
- ❖ CHAIN 控件：图片连播控件，用于一组图片的连续播放。
- ❖ PROGRESSBAR 控件：进度条控件，专门用于音量条、进度条的显示。
- ❖ SPECTRUM 控件：频谱控件，专门用于播放器频谱显示的控件。
- ❖ SELECT 控件：浏览器控件，用于表格生成维护 and 管理的控件。

上述控件均是可选配的，用户可以通过\Bsp\EVK_V2.0\source_macro.h 中特定的预编译宏加载或移除相关控件。控件的其具体实现，可参见\Gui 中的相关源码。

2.6.1 GUI_GcCreate

函数原型	GUI_CONTROL_CLASS * GUI_GcCreate(eGc eGcId, void* arg)
eGcId	控件类型
arg	创建参数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据控件类型，调用控件初始化函数。

2.6.2 GUI_GcDelete

函数原型	rk_err_t GUI_GcDelete(HGC hGc)
hGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据控件类型，调用控件反初始化函数

2.6.3 GUI_GcDelete

函数原型	rk_err_t GUI_GcDelete(HGC hGc)
hGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据控件类型，调用控件反初始化函数

2.6.4 GUI_OperWidge

函数原型	rk_err_t GUI_OperWidge(HGC hGc, eRKGUI_OPERATE_CMD cmd, void* arg)
hGc	控件句柄
cmd	操作命令: OPERATE_DISPLAY: 控件显示 OPERATE_SET_CONTENT: 修改内容 OPERATE_SET_DISPLAY: 显示/隐藏 OPERATE_START_MOVE: 使能动画效果
arg	带入参数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据控件类型，调用控件反初始化函数

2.6.5 GcDisplay

函数原型	rk_err_t GcDisplay(GUI_CONTROL_CLASS * pGc)
hGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据控件类型，调用控件显示函数

2.7 消息与焦点管理器

UI 管理器，除负责响应应用程序发来的请求事件外，还负责对输入设备发来的事件进行中转。应用程序可以通过 `GuiTask_AppRecvMsg` 和 `GuiTask_UnAppRecvMsg` 函数来申请接收或停止接收这些信息。如果，当前系统不存在焦点，UI 管理器则会通过回调函数将这些事件通知应用程序。而如果，当前存在活跃的焦点控件，那么 UI 管理器将把事件优先发往焦点控件。通常焦点控件只会接收那些自己能够处理的信息，对于无法解析或无关的事件。UI 管理器将会把它们转发给应用程序。

2.7.1 GuiTask_AppRecvMsg

函数原型	<code>rk_err_t GuiTask_AppRecvMsg(P_APP_RECV_MSG pApp)</code>
pApp	回调函数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	用于 APP 向 GUITask 申请接收输入事件消息。

2.7.2 GuiTask_AppUnRecvMsg

函数原型	<code>rk_err_t GuiTask_AppUnRecvMsg(P_APP_RECV_MSG pApp)</code>
pApp	回调函数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	用于 APP 向 GUITask 申请停止接收输入事件消息。

2.7.3 GuiTask_Setfocus

函数原型	<code>rk_err_t GuiTask_Setfocus(HGC pGc)</code>
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功

	RK_ERROR: 执行错误
功能说明	将指定控件设置为焦点。

2.7.4 GuiTask_Delfocus

函数原型	rk_err_t GuiTask_Delfocus(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	取消指定控件为焦点。

2.8 GUI API 介绍

2.8.1 GUITask_CreateWidget

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	控件类型
arg	创建参数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	此函数用于 APP 向用户管理申请创建一个控件, 并将控件加入控件列表管理。

2.8.2 GuiTask_DeleteWidget

函数原型	rk_err_t GuiTask_DeleteWidget(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	调用 GuiTask_DeleteWidget 后, 控件管理会负责将控件所使用的资源进行回收, 并将控件句柄从图层管理链表中移除。

	值得注意的是：删除控件后，若未达成刷新 LCD 条件，控件不会从 LCD 上“消失”。用户若期望达成“消失”效果，可在删除前对控件设置隐藏。
--	--

2.8.3 GuiTask_OperWidget

函数原型	rk_err_t GuiTask_OperWidget(HGC pGc, eRKGUI_OPERATE_CMD cmd, void *arg, uint32 mode)
pGc	控件句柄
cmd	命令参数
arg	带入参数
mode	SYNC_MODE: 同步模式 ASYNC_MODE: 异步模式
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

2.8.4 GuiTask_ScreenLock

函数原型	rk_err_t GuiTask_ScreenLock(void)
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示器锁定函数：此函数用于锁定显示器。当显示器处于锁定状态时，图像只会更新到 Framebuffer 上，而不对 LCD 进行操作。以此防止系统处理能力不足时，造成的残影问题。

2.8.5 GuiTask_ScreenUnlock

函数原型	rk_err_t GuiTask_ScreenLock(void)
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示器解锁函数：此函数用于解锁显示器。锁定与解锁函数必须成对出现。

2.8.6 GuiTask_Setfocus

函数原型	rk_err_t GuiTask_Setfocus(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	将焦点移到指定的控件上。

2.8.7 GuiTask_Delfocus

函数原型	rk_err_t GuiTask_Delfocus(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	移除特定控件上的焦点。移除焦点后，活跃焦点自动返回上一个焦点。

2.8.8 GuiTask_AppReciveMsg

函数原型	rk_err_t GuiTask_AppReciveMsg(P_APP_RECVIE_MSG pApp)
pApp	回调函数地址
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	回调注册函数。此函数用于 APP 向 GUI 请求接收消息。当按键或焦点控件有事件产生时，将会通过回调函数通知 APP 处理。 注意：当回调函数发生时，程序处于 GUI 线程中。此时，严禁 APP 再次请求操作 GUI。否则将有可能造成重入，导致线程锁死。建议的做法是通过 OS 提供的消息队列机制，将事件发送到 APP 线程中处理。

2.8.9 GuiTask_UnAppReciveMsg

函数原型	rk_err_t GuiTask_AppUnReciveMsg(P_APP_RECVIE_MSG pApp)
pApp	回调函数地址
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	反注册回调函数。此函数用于 APP 向 GUI 请求停止接收消息。

3 ICON 控件

3.1 ICON 控件简介

ICON 控件，即图标管理控件。该控件是对固件中图片资源访问方法的封装。用户可以通过 ICON 控件显示一个固件中的图片。

3.2 ICON 创建

函数原型	HGC GuiTask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_ICON（创建 ICON 控件时）
arg	RKGUI_ICON_ARG 结构（创建 ICON 控件时）
返回值	控件句柄

3.2.1 RKGUI_ICON_ARG

```
typedef struct
{
    int display;
    uint32 resource;
    int x;
    int y;
    int level;
}RKGUI ICON ARG;
```

名称	RKGUI_ICON_ARG
display	隐藏标志，0 隐藏，1 显示。
resource	图标 SID，见\Bsp\EVK_V2.0\Resource\ImageResourceID.h。
x	X 轴显示坐标
y	Y 轴显示坐标
level	图层，0 底层，1 顶层（即状态栏）。

3.2.2 参考例程

```

RKGUI_ICON_ARG pstIconArg;
pstIconArg.x= 0;
pstIconArg.y= 0;
pstIconArg.display= 1;
pstIconArg.resource= IMG_ID_MUSIC_BACKGROUND;
pstIconArg.level= 0;
hGc= GuiTask_CreateWidget(GUI_CLASS_ICON, &pstIconArg);

```

3.3 ICON 内容变更

函数原型	rk_err_t GuiTask_OperWidget(HGC pGc, eRKGUI_OPERATE_CMD cmd, void *arg, uint32 mode)
pGc	控件句柄
cmd	OPERATE_SET_CONTENT。（替换内容时）
arg	定义在 ImageResourceID.h 中的，图片资源编号。（替换内容时）
mode	SYNC_MODE：同步模式 ASYNC_MODE：异步模式
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误

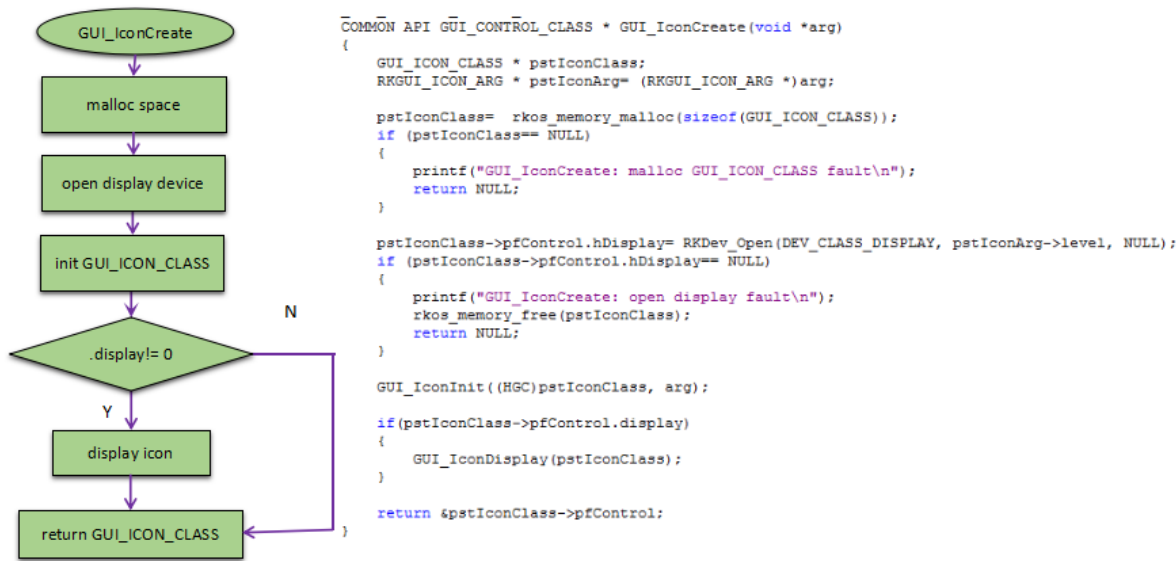
3.3.1 变更图标

```

GuiTask_OperWidget( hGc,  OPERATE_SET_CONTENT,
                    (void *)IMG_ID_USB_BACKGROUND,  SYNC_MODE);

```

3.4 ICON 创建流程



当 GUITask 接收到 ICON 控件创建请求时，将通过控件管理器调用 GUI_IconCreate 函数进行控件创建。首先，调用 rkos_memory_malloc 为 ICON 控件类分配内存空间。然后，根据用户传入的图层参数，打开对应的 Display Device。最后，通过 GUI_IconInit 函数对 ICON 控件类的各项参数进行初始化，并根据用户传入的 Display 参数决定是否显示。

3.5 实现说明

3.5.1 GUI_ICON_CLASS

```
typedef struct _GUI_ICON_CLASS
[
    GUI_CONTROL_CLASS pfControl;
    PICTURE_INFO_STRUCT psPictureInfo;
]GUI_ICON_CLASS;
```

名称	GUI_ICON_CLASS
pfControl	控件类，包含了控件的基本信息。
psPictureInfo	图片信息类，包含图片在固件中的偏移地址、位置、大小等信息。

```
typedef struct PictureInfoStruct
{
    UINT16  xSize;
    UINT16  ySize;
    UINT16  x;
    UINT16  y;
    UINT32  totalSize;
    UINT32  offsetAddr;

} PICTURE_INFO_STRUCT;
```

名称	PICTURE_INFO_STRUCT
xSize	图片 x 大小。
ySize	图片 y 大小。
x	图片 x 轴坐标
y	图片 y 轴坐标
totalSize	图片总字节数
offsetAddr	图片固件偏移地址。

3.5.2 GUI_IconCreate

函数原型	GUI_CONTROL_CLASS * GUI_IconCreate(void *arg)
arg	创建参数, RKGUI_ICON_ARG 类型
返回值	NULL: 创建失败 非 NULL: 控件句柄
功能说明	根据创建参数, 创建 ICON 控件, 并返回控件句柄。

3.5.3 GUI_IconDelete

函数原型	rk_err_t GUI_IconDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据句柄回收控件资源。

3.5.4 GUI_IconSetID

函数原型	rk_err_t GUI_IconSetID(HDC pGc, uint32 ResourceId)
pGc	控件句柄
ResourceId	图标 SID 编号
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据 SID, 初始化大小位置信息。

3.5.5 GUI_IconInit

函数原型	rk_err_t GUI_IconInit(HDC pGc, void * arg)
pGc	控件句柄
arg	创建参数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据创建参数, 初始化 GUI_ICON_CLASS 结构。

3.5.6 GUI_IconDisplay

函数原型	rk_err_t GUI_IconDisplay(HDC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示控件。

3.5.7 GUI_IconDisplayResource

函数原型	rk_err_t GUI_IconDisplayResource(HDC hDisplay, int x0, int y0, int x1, int y1, PICTURE_INFO_STRUCT * psPictureInfo)
------	---

hDisplay	Display 设备句柄
x0	x 轴起始坐标。
y0	y 轴起始坐标。
x1	宽度： >=0: 指定大小 < 0: 自动设置
y1	长度： >=0: 指定大小 < 0: 自动设置
psPictureInfo	图片信息结构
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据图片信息结构，在 (x0, y0) 位置，刷新 (x1, y1) 大小的图片。

3.5.8 GUI_IconAreaUpdata

函数原型	rk_err_t GUI_IconAreaUpdata(HDC hDisplay, int x, int y, int xSize, int ySize, int bckx, int bcky, PICTURE_INFO_STRUCT * psPictureInfo)
hDisplay	Display 设备句柄
x	x 轴起始坐标。
y	y 轴起始坐标。
xSize	宽度： >=0: 指定大小 < 0: 自动设置
ySize	长度： >=0: 指定大小 < 0: 自动设置
bckx	图标 x 轴偏移
bcky	图片 y 轴偏移

psPictureInfo	图片信息结构
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据图片信息结构，在（x，y）位置，从图片（bckx，bcky）起始位置，刷新（xSize，ySize）大小的图片。

4 IMAGE 控件

4.1 IMAGE 控件简介

ICON 控件，即影像管理控件。该控件具有矩形框、填充、矢量三种模式。当控件处于矩形框模式时，能够高效地绘制和管理一个纯色实心矩形框。当控件处于填充模式时，能够高效的将用户一组 RGB 数据显示在 LCD 上。当控件处在矢量模式时，用户可以用它在 LCD 的指定位置打点，以此绘制特定的矢量图形。

4.2 IMAGE 创建

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_IMAGE（创建 IMAGE 控件时）
arg	RKGUI_IMAGE_ARG 结构（创建 IMAGE 控件时）
返回值	控件句柄

4.2.1 RKGUI_IMAGE_ARG

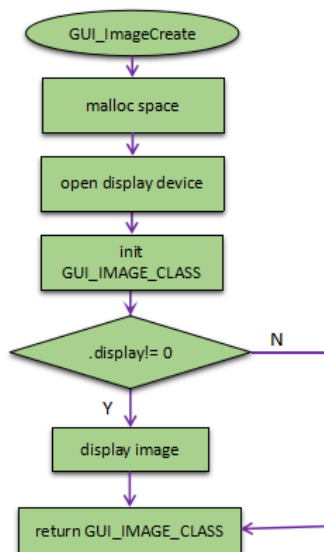
```
typedef struct
{
    int display;
    int level;
    eIMAGE_DISPLAY_CMD cmd;
    LUCENCY_TYPE lucency;
    int x;
    int y;
    int xSize;
    int ySize;
    void * buf;
    uint8 R;
    uint8 G;
    uint8 B;
    uint8 transparency;
}RKGUI_IMAGE_ARG;
```

名称	RKGUI_IMAGE_ARG
display	隐藏标志，0 隐藏，1 显示。
level	图层，0 底层，1 顶层（即状态栏）。
cmd	模式选择： IMAGE_CMD_IMAGE：填充模式（尚不支持） IMAGE_CMD_RECT：矩形框模式 IMAGE_CMD_PIXEL：像素模式（尚不支持）
lucency	透明模式： LUCENCY：透明模式 OPACITY：非透明模式
x	X 轴显示坐标
y	Y 轴显示坐标
xSize	宽度
ySize	长度
buf	填充数据（仅填充模式有意义）
R、G、B	红绿蓝颜色分量 0~255（仅像素模式和矩形框模式有效）
transparency	透明分量 0~255（仅像素模式和矩形框模式有效）

4.2.2 矩形框模式

```
RKGUI_IMAGE_ARG pstImageArg;  
pstImageArg.cmd= IMAGE_CMD_RECT;  
pstImageArg.R= 0;  
pstImageArg.G= 0xFF;  
pstImageArg.B= 0;  
pstImageArg.transparency= 0;  
pstImageArg.x= 100;  
pstImageArg.y= 30;  
pstImageArg.xSize= 5;  
pstImageArg.ySize= 90;  
pstImageArg.display = 0;  
pstImageArg.level= 0;  
pstImageArg.lucency= OPACITY;  
hGc= GUITask_CreateWidget(GUI_CLASS_IMAGE, &pstImageArg);
```

4.3 IMAGE 创建流程



当 GUITask 接收到 IMAGE 控件创建请求时,将通过控件管理器调用 GUI_ImageCreate 函数进行控件创建。首先,调用 rkos_memory_malloc 为 IMAGE 控件类分配内存空间。然后,根据用户传入的图层参数,打开对应的 Display Device。最后,通过对 IMAGE 控件类的各项参数进行初始化,并根据用户传入的 Display 参数决定是否显示。

```
COMMON API GUI_CONTROL_CLASS * GUI_ImageCreate(void *arg)
{
    GUI_IMAGE_CLASS * pstImageClass;
    RKGUI_IMAGE_ARG * pstImageArg= (RKGUI_IMAGE_ARG *)arg;;

    pstImageClass= rkos_memory_malloc(sizeof(GUI_IMAGE_CLASS));           // malloc space
    if (pstImageClass== NULL)
    {
        return NULL;
    }

    // open display device
    pstImageClass->pfControl.hDisplay= RKDev_Open(DEV_CLASS_DISPLAY, pstImageArg->level, NULL);
    if (pstImageClass->pfControl.hDisplay== NULL)
    {
        printf("Create Image: open display fault\n");
        rkos_memory_free(pstImageClass);
        return NULL;
    }

    // init GUI_IMAGE_CLASS
    pstImageClass->pfControl.ClassID = GUI_CLASS_IMAGE;
    pstImageClass->cmd= pstImageArg->cmd;
    pstImageClass->pfControl.x0 = pstImageArg->x;
    pstImageClass->pfControl.y0 = pstImageArg->y;
    pstImageClass->pfControl.xSize = pstImageArg->xSize;
    pstImageClass->pfControl.ySize = pstImageArg->ySize;
```

```
pstImageClass->pfControl.xSize = pstImageArg->xSize;
pstImageClass->pfControl.ySize = pstImageArg->ySize;
pstImageClass->pfControl.rotate= ROTATE_NOMAL;
pstImageClass->pfControl.level= pstImageArg->level;

pstImageClass->buf = pstImageArg->buf;
pstImageClass->R = pstImageArg->R;
pstImageClass->G = pstImageArg->G;
pstImageClass->B = pstImageArg->B;
pstImageClass->transparency= pstImageArg->transparency;
pstImageClass->pfControl.display= pstImageArg->display;
pstImageClass->pfControl.flag= 0;

pstImageClass->pfControl.lucency= pstImageArg->lucency;
pstImageClass->pfControl.translucence= 0;

// display image
if( pstImageClass->pfControl.display )
{
    GUI_ImageDisplay(pstImageClass);
}
return &pstImageClass->pfControl;
}
```

4.4 实现说明

4.4.1 GUI_IMAGE_CLASS

```
typedef struct _GUI_IMAGE_CLASS
{
    GUI_CONTROL_CLASS pfControl;
    eIMAGE_DISPLAY_CMD cmd;
    LUCENCY_TYPE lucency;

    void * buf;
    uint8 R;
    uint8 G;
    uint8 B;
    uint8 transparency;
}GUI_IMAGE_CLASS;
```

名称	GUI_IMAGE_CLASS
pfControl	控件类，包含了控件的基本信息。
cmd	模式选择： IMAGE_CMD_IMAGE：填充模式 IMAGE_CMD_RECT：矩形框模式 IMAGE_CMD_PIXEL：像素模式
lucency	透明模式： LUCENCY：透明模式

	OPACITY: 非透明模式
buf	填充数据（仅填充模式有效）
R、G、B	红绿蓝颜色分量 0~255（仅像素模式和矩形框模式有效）
transparency	透明分量 0~255（仅像素模式和矩形框模式有效）

4.4.2 GUI_ImageCreate

函数原型	GUI_CONTROL_CLASS * GUI_ImageCreate(void *arg)
arg	创建参数, RKGUI_IMAGE_ARG 类型
返回值	NULL: 创建失败 非 NULL: 控件句柄
功能说明	根据创建参数, 创建控件, 并返回控件句柄。

4.4.3 GUI_ImageDelete

函数原型	rk_err_t GUI_ImageDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据句柄回收控件资源。

4.4.4 GUI_ImageDisplay

函数原型	rk_err_t GUI_ImageDisplay(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示控件。

4.4.5 GUI_ImageDisplayRect

函数原型	rk_err_t GUI_ImageDisplayRect(HDC hDisplay, int x, int y, int xSize, int ySize, int R, int G, int B, int transparency)
pGc	Display 设备句柄
x	X 轴显示位置
y	Y 轴显示位置
xSize	矩形框宽度
ySize	矩形框长度
R、G、B	红绿蓝颜色分量 0~255
transparency	透明分量 0~255
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	在 (x, y) 处显示大小为 (xSize, ySize) 颜色为 RGB 的纯色矩形框。

5 TEXT 控件

5.1 TEXT 控件简介

TEXT 控件，即文字管理控件。该控件具有普通模式和 SID 模式两种。正常模式用于将用户传入的特定 unicode 码字符串，转为点阵图形并显示。SID 模式下用户可以将特定的文字信息存入固件资源中，并通过对应的 SID 编号进行访问。

5.2 TEXT 创建

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_TEXT（创建 TEXT 控件时）
arg	RKGUI_TEXT_ARG 结构（创建 TEXT 控件时）
返回值	控件句柄

5.2.1 RKGUI_TEXT_ARG

```
typedef struct _RKGUI_TEXT_ARG
{
    eTEXT_DISPLAY_CMD cmd;
    int align;

    int display;
    int level;
    int resource;
    void * text;
    int x;
    int y;
    int xSize;
    int ySize;

    LUCENCY_TYPE lucency;
    int Backdrop;
    int BackdropX;
    int BackdropY;

    int ForegroundR;
    int ForegroundG;
    int ForegroundB;

    int BackgroundR;
    int BackgroundG;
    int BackgroundB;

    eTEXT_OPERATE_CMD opercmd;
}RKGUI_TEXT_ARG;
```

名称	RKGUI_TEXT_ARG
cmd	模式选择。 TEXT_CMD_BUF : 普通模式 TEXT_CMD_ID: SID 模式
align	对齐方式: TEXT_ALIGN_X_LEFT: X 轴左对齐 TEXT_ALIGN_X_RIGHT: X 轴右对齐 TEXT_ALIGN_X_CENTER: X 轴居中对齐 TEXT_ALIGN_Y_UP: Y 轴顶部对齐 TEXT_ALIGN_Y_DOWN: Y 轴底部对齐 TEXT_ALIGN_Y_CENTER: Y 轴居中对齐
display	隐藏标志, 0 隐藏, 1 显示。
resource	资源 SID (仅 SID 模式下有效)
text	显示内容 (unicode 码, 仅普通模式下有效)
x	X 轴显示坐标

y	Y 轴显示坐标
xSize	文本框宽度。< 0 自动填写
ySize	文本框高度。< 0 自动填写
lucency	透明模式： LUCENCY：透明模式 OPACITY：非透明模式
Backdrop	背景图片 SID（仅非透明模式下有效）。< 0 使用纯色背景
BackdropX	背景 x 偏移
BackdropY	背景 y 偏移
ForegroundR、 ForegroundG、 ForegroundB	取值范围 0~255，字体颜色。
BackgroundR、 BackgroundG、 BackgroundB	取值范围 0~255，背景颜色。（仅非透明模式，且 Backdrop< 0 时有效）
opercmd	操作模式选择： TEXT_SET_CONTENT：改变显示内容 TEXT_SET_COLOUR：改变字体颜色 TEXT_SET_ALIGN：改变对齐方式 TEXT_SET_BACKGROUND：改变背景图片

5.2.2 普通模式

```
HGC hGc;
RKGUI_TEXT_ARG pstTextArg;
pstTextArg.cmd= TEXT_CMD_BUF;           // 普通模式
pstTextArg.align= TEXT_ALIGN_X_LEFT;    // x 轴左对齐
pstTextArg.x= 20;                       // 显示位置 x,y(20,50)
pstTextArg.y= 50;
pstTextArg.xSize= 100;                   // 文本框大小 100*50 像素
pstTextArg.ySize= 50;

pstTextArg.lucency= OPACITY;             // 非透明显示模式
```



```
pstTextArg.Backdrop= IMG_ID_POWERON0; // 文本框背景图标（非透明模式下有效）
pstTextArg.ForegroundR= 0; // 前景色，即字体颜色
pstTextArg.ForegroundG= 0;
pstTextArg.ForegroundB= 0;

pstTextArg.BackgroundR= 255; // 背景色
pstTextArg.BackgroundG= 255;
pstTextArg.BackgroundB= 255;
pstTextArg.BackdropX= 20; // 文本框位于图标的
pstTextArg.BackdropY= 50;

pstTextArg.text= TEXT("TEXT 控件");
pstTextArg.display = 1; // 创建即显示
pstTextArg.level= 0; // 图层 0
hGc= GUITask_CreateWidget(GUI_CLASS_TEXT, &pstTextArg);
```

5.2.3 SID 模式

```
HGC hGc;
RKGUI_TEXT_ARG pstTextArg;
pstTextArg.cmd= TEXT_CMD_ID; // SID 模式
pstTextArg.align= TEXT_ALIGN_X_LEFT; // x 轴左对齐
pstTextArg.x= 20; // 显示位置 x,y(20,50)
pstTextArg.y= 50;
pstTextArg.xSize= 100; // 文本框大小 100*50 像素
pstTextArg.ySize= 50;

pstTextArg.lucency= OPACITY; // 非透明显示模式
pstTextArg.Backdrop= IMG_ID_POWERON0; // 文本框背景图标（非透明模式下有效）
pstTextArg.ForegroundR= 0; // 前景色，即字体颜色
pstTextArg.ForegroundG= 0;
pstTextArg.ForegroundB= 0;

pstTextArg.BackgroundR= 255; // 背景色
pstTextArg.BackgroundG= 255;
pstTextArg.BackgroundB= 255;
pstTextArg.BackdropX= 20; // 文本框位于图标的
pstTextArg.BackdropY= 50;
pstTextArg.resource= SID_MUSIC;
pstTextArg.display = 1;
pstTextArg.level= 0;
hGc= GUITask_CreateWidget(GUI_CLASS_TEXT, &pstTextArg);
```

5.3 TEXT 内容变更

函数原型	rk_err_t GuiTask_OperWidget(HGC pGc, eRKGUI_OPERATE_CMD cmd, void *arg, uint32 mode)
pGc	控件句柄
cmd	OPERATE_SET_CONTENT。（替换内容时）
arg	RKGUI_TEXT_ARG 具体意义与命令参数有关。
mode	SYNC_MODE: 同步模式 ASYNC_MODE: 异步模式
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

5.3.1 变更文字

```
RKGUI_TEXT_ARG pstTextArg;
pstTextContent.opercmd= TEXT_SET_CONTENT;
pstTextContent.resource = SID_RECORD_PAUSE;
GuiTask_OperWidget(hGc, OPERATE_SET_CONTENT, &pstTextContent, SYNC_MODE);
```

5.3.2 变更字体颜色

```
RKGUI_TEXT_ARG pstTextArg;
pstTextContent.opercmd= TEXT_SET_COLOUR;
pstTextContent.ForegroundR = 0xFF;
pstTextContent.ForegroundG = 0;
pstTextContent.ForegroundB = 0;
GuiTask_OperWidget(hGc, OPERATE_SET_CONTENT, &pstTextContent, SYNC_MODE);
```

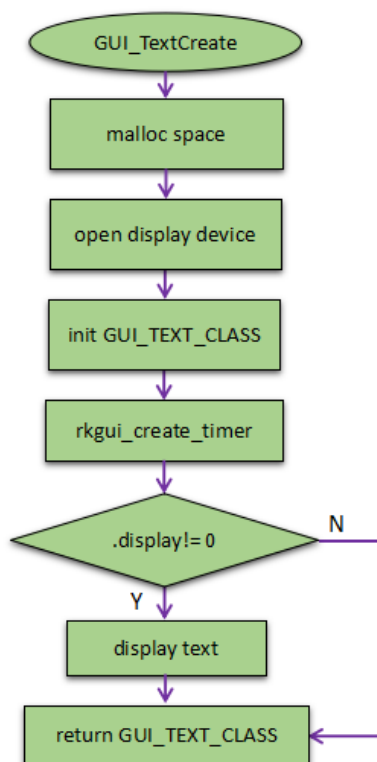
5.3.3 变更背景图片/背景色

```
RKGUI_TEXT_ARG pstTextArg;
pstTextArg.opercmd= TEXT_SET_BACKGROUND;
pstTextArg.Backdrop = IMG_ID_MUSIC_BACKGROUND;
pstTextArg.BackdropX= 20;
pstTextArg.BackdropY= 50;
GuiTask_OperWidget(hGc, OPERATE_SET_CONTENT, &pstTextArg, SYNC_MODE);
```

5.3.4 变更对齐方式

```
RKGUI_TEXT_ARG pstTextArg;  
pstTextArg.opercmd= TEXT_SET_ALIGN;  
pstTextArg.align = TEXT_ALIGN_X_CENTER | TEXT_ALIGN_Y_CENTER; // 居中对齐  
GuiTask_OperWidget(hGc, OPERATE_SET_CONTENT, &pstTextArg, SYNC_MODE);
```

5.4 TEXT 创建流程



当 GUITask 接收到 TEXT 控件创建请求时，将通过控件管理器调用 GUI_ImageCreate 函数进行控件创建。首先，调用 rkos_memory_malloc 为 TEXT 控件类分配内存空间。然后，根据用户传入的图层参数，打开对应的 Display Device。再然后，通过 rkgui_create_timer 函数申请创建一个 UI 定时器，完成用于走马灯效果。最后，初始化 TEXT 控件类参数，并根据 display 决定是否立即显示。

值得注意的是，TEXT_CMD_ID 模式在初始化时，将通过 GUI_TextExtractString 函数从固件中取得字符串信息。在用户使用自动计算字符串宽度的情况下，还需要调用 GUI_TextExtractSize 来获取字符串实际长度及 GUI_TextAutoAlign 来计算字符串的起始坐标。

```
COMMON API GUI_CONTROL_CLASS * GUI_TextCreate(void *arg)
{
    GUI_TEXT_CLASS * pstTextClass;
    RKGUI_TEXT_ARG * pstTextArg= (RKGUI_TEXT_ARG *)arg;

    pstTextClass= rkos_memory_malloc(sizeof(GUI_TEXT_CLASS));
    if (pstTextClass== NULL)
    {
        return NULL;
    }

    pstTextClass->pfControl.hDisplay= RKDev_Open(DEV_CLASS_DISPLAY, pstTextArg->level, NULL);
    if (pstTextClass->pfControl.hDisplay== NULL)
    {
        printf("Create Text: open display fault\n");
        rkos_memory_free(pstTextClass);
        return NULL;
    }

    pstTextClass->pfControl.ClassID = GUI_CLASS_TEXT;
    pstTextClass->pfControl.x0= pstTextArg->x;
    pstTextClass->pfControl.y0= pstTextArg->y;
    pstTextClass->pfControl.xSize= pstTextArg->xSize;
    pstTextClass->pfControl.ySize= pstTextArg->ySize;

    if(pstTextClass->pfControl.xSize <= 0)
        pstTextClass->xAuto= 1;
    else
        pstTextClass->xAuto= 0;

    if(pstTextClass->pfControl.ySize <= 0)
        pstTextClass->yAuto= 1;
    else
        pstTextClass->yAuto= 0;

    pstTextClass->pfControl.display= pstTextArg->display;
    pstTextClass->pfControl.rotate= ROTATE_NOMAL;
    pstTextClass->pfControl.level= pstTextArg->level;

    pstTextClass->pfControl.lucency= pstTextArg->lucency;
    pstTextClass->Backdrop = pstTextArg->Backdrop;
    pstTextClass->BackdropX= pstTextArg->BackdropX;
    pstTextClass->BackdropY= pstTextArg->BackdropY;

    pstTextClass->ForegroundR = pstTextArg->ForegroundR;
    pstTextClass->ForegroundG = pstTextArg->ForegroundG;
    pstTextClass->ForegroundB = pstTextArg->ForegroundB;

    pstTextClass->BackgroundR = pstTextArg->BackgroundR;
    pstTextClass->BackgroundG = pstTextArg->BackgroundG;
    pstTextClass->BackgroundB = pstTextArg->BackgroundB;

    pstTextClass->cmd= pstTextArg->cmd;

    pstTextClass->font= TYPE12X12;
    pstTextClass->text= NULL;
    pstTextClass->text_bck= NULL;

    pstTextClass->pfControl.flag= 0;
    pstTextClass->pfControl.lucency= pstTextArg->lucency;
    pstTextClass->pfControl.translucence= 0;
    pstTextClass->align= pstTextArg->align;

    if(pstTextArg->resource>= 0 && pstTextArg->cmd== TEXT_CMD_ID)
    {
        pstTextClass->text= GUI_TextExtractString(pstTextArg->resource);
        pstTextClass->text_bck= pstTextClass->text;
        if( pstTextClass->xAuto )           // Auto Fill xSize
        {
            pstTextClass->pfControl.xSize = GUI_TextExtractSize(pstTextClass->font, pstTextClass->text);
        }
        if( pstTextClass->yAuto )           // Auto Fill ySize
        {
            pstTextClass->pfControl.ySize = 12;
        }
    }
}
```

```
GUI_TextAutoAlign(pstTextClass->font, pstTextClass->align,
    pstTextClass->pfControl.x0, pstTextClass->pfControl.y0,
    pstTextClass->pfControl.xSize, pstTextClass->pfControl.ySize,
    &pstTextClass->StartX, &pstTextClass->StartY,
    &pstTextClass->xSize, &pstTextClass->ySize,
    pstTextClass->text);
}
else if(pstTextClass->pfControl.display && pstTextArg->text!= NULL)
{
    pstTextClass->text = rkos_memory_malloc(StrLenW(pstTextArg->text) * 2 + 2);

    pstTextClass->text_bck = pstTextClass->text;

    memcpy(pstTextClass->text, pstTextArg->text, StrLenW(pstTextArg->text) * 2 + 2);

    if( pstTextClass->xAuto )
    {
        pstTextClass->pfControl.xSize = GUI_TextExtractSize(pstTextClass->font, pstTextClass->text);
    }
    if( pstTextClass->yAuto )
    {
        pstTextClass->pfControl.ySize = 12;
    }

    GUI_TextAutoAlign(pstTextClass->font, pstTextClass->align,
        pstTextClass->pfControl.x0, pstTextClass->pfControl.y0,
        pstTextClass->pfControl.xSize, pstTextClass->pfControl.ySize,
        &pstTextClass->StartX, &pstTextClass->StartY,
        &pstTextClass->xSize, &pstTextClass->ySize,
        pstTextClass->text);
}

rkgui_create_timer(pstTextClass, GUI_TextTimer);

if(pstTextClass->pfControl.display)
{
    GUI_TextDisplay(pstTextClass);
}
return &pstTextClass->pfControl;
}
```

5.5 实现说明

5.5.1 GUI_TEXT_CLASS

```
typedef struct _GUI_TEXT_CLASS
{
    GUI_CONTROL_CLASS pfControl;
    eTEXT_DISPLAY_CMD cmd;
    int align;

    int StartX;
    int StartY;
    int xSize;
    int ySize;

    int xAuto;
    int yAuto;

    int Backdrop;
    int BackdropX;
    int BackdropY;

    int ForegroundR;
    int ForegroundG;
    int ForegroundB;

    int BackgroundR;
    int BackgroundG;
    int BackgroundB;

    FONT_TYPE font;
```

```
int speed;

Ucs2 * text;
Ucs2 * text_bck;
}GUI_TEXT_CLASS;
```

名称	GUI_TEXT_CLASS
pfControl	控件类，包含了控件的基本信息。
cmd	模式选择。 TEXT_CMD_BUF：普通模式 TEXT_CMD_ID: SID 模式
align	对齐方式： TEXT_ALIGN_X_LEFT: X 轴左对齐 TEXT_ALIGN_X_RIGHT: X 轴右对齐 TEXT_ALIGN_X_CENTER: X 轴居中对齐 TEXT_ALIGN_Y_UP: Y 轴顶部对齐 TEXT_ALIGN_Y_DOWN: Y 轴底部对齐 TEXT_ALIGN_Y_CENTER: Y 轴居中对齐
StartX	用于记录文字的显示起始坐标
StartY	用于记录文字的显示起始坐标
xSize	用于记录文字宽度
ySize	用于记录文字高度
xAuto	用于标识是否需要由软件指定文字宽度。 1 软件计算 0 用户指定
yAuto	用于标识是否需要由软件指定文字高度。 1 软件计算 0 用户指定
Backdrop	背景图片 SID（仅非透明模式下有效）。< 0 使用纯色背景
BackdropX	背景 x 偏移
BackdropY	背景 y 偏移
ForegroundR、	取值范围 0~255，字体颜色。

ForegroundG、 ForegroundB	
BackgroundR、 BackgroundG、 BackgroundB	取值范围 0~255，背景颜色。（仅非透明模式，且 Backdrop< 0 时有效）
font	字体： TYPE12X12（默认）：12*12 TYPE16X16：16*16
speed	用于调节文件滚动快慢
text	用于存放当前字符串
text_bck	滚动效果时，用于存放原始字符串

5.5.2 GUI_TextCreate

函数原型	GUI_CONTROL_CLASS * GUI_TextCreate(void *arg)
arg	创建参数，RKGUI_TEXT_ARG 类型
返回值	NULL：创建失败 非 NULL：控件句柄
功能说明	根据创建参数，创建控件，并返回控件句柄。

5.5.3 GUI_TextDelete

函数原型	rk_err_t GUI_TextDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	根据句柄回收控件资源。

5.5.4 GUI_TextDisplay

函数原型	rk_err_t GUI_TextDisplay(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示控件。

5.5.5 GUI_TextSetContent

函数原型	rk_err_t GUI_TextSetContent(HGC pGc, void * arg)
pGc	控件句柄
arg	带入参数, RKGUI_TEXT_ARG 结构,
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据 opercmd 参数, 修改控件显示。 TEXT_SET_BACKGROUND: 修改背景 TEXT_SET_COLOUR: 修改字体颜色 TEXT_SET_CONTENT: 修改显示文字 TEXT_SET_ALIGN: 修改对齐方式 透明模式: 将会造成所有遮叠的控件进行刷新。 非透明模式: 根据 SID 刷新背景区域, 然后显示文字。

5.5.6 GUI_TextSetContent

函数原型	rk_err_t GUI_TextSetContent(HGC pGc, void * arg)
pGc	控件句柄
arg	带入参数, RKGUI_TEXT_ARG 结构,
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

功能说明	根据 opercmd 参数，修改控件显示。 TEXT_SET_BACKGROUND: 修改背景 TEXT_SET_COLOUR: 修改字体颜色 TEXT_SET_CONTENT: 修改显示文字 TEXT_SET_ALIGN: 修改对齐方式 透明模式: 将会造成所有遮叠的控件进行刷新。 非透明模式: 根据 SID 刷新背景区域，然后显示文字。
------	--

5.5.7 GUI_TextDisplayID

函数原型	rk_err_t GUI_TextDisplayID(HDC hDisplay, int x0, int y0, int x1, int y1, int id, FONT_TYPE font, int R, int G, int B, int transparency)
hDisplay	Display 设备句柄
x0	显示位置
y0	显示位置
x1	文字宽度。 < 0 自动计算 > 文字真实宽度，按文字真实宽度显示 < 文字真实宽度，按指定宽度显示，丢弃超出部分
y1	文字高度。 < 0 自动计算 > 文字真实高度，按文字真实高度显示 < 文字真实高度，按指定高度显示，丢弃超出部分
id	文字 SID
font	字体: TYPE12X12: 12*12 TYPE16X16: 16*16
RGB、transparency	字体颜色
返回值	RK_SUCCESS: 执行成功

	RK_ERROR: 执行错误
功能说明	根据指定的 id, 从固件中取得字符串并显示。

5.5.8 GUI_TextDisplayBuff

函数原型	rk_err_t GUI_TextDisplayBuff(HDC hDisplay, int x0, int y0, int x1, int y1, Ucs2 * text, FONT_TYPE font, int R, int G, int B, int transparency)
hDisplay	Display 设备句柄
x0	显示位置
y0	显示位置
x1	文字宽度。 < 0 自动计算 > 文字真实宽度, 按文字真实宽度显示 < 文字真实宽度, 按指定宽度显示, 丢弃超出部分
y1	文字高度。 < 0 自动计算 > 文字真实高度, 按文字真实高度显示 < 文字真实高度, 按指定高度显示, 丢弃超出部分
text	Unicode 字符串
font	字体: TYPE12X12: 12*12 TYPE16X16: 16*16
RGB、transparency	字体颜色
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示指定的字符串。

5.5.9 GUI_BmpFronDisplay

函数原型	rk_err_t GUI_BmpFronDisplay(HDC hDisplay, void *pbuf,int x, int y, int xSize,
------	---

	int ySize, int R, int G, int B, int transparency)
hDisplay	Display 设备句柄
pbuf	文字点阵数据
x0	显示位置
y0	显示位置
xSize	文字宽度。 < 0 自动计算 > 文字真实宽度，按文字真实宽度显示 < 文字真实宽度，按指定宽度显示，丢弃超出部分
ySize	文字高度。 < 0 自动计算 > 文字真实高度，按文字真实高度显示 < 文字真实高度，按指定高度显示，丢弃超出部分
RGB、transparency	字体颜色
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	将文字的点阵数据，在指定的 x,y 位置上显示。

5.5.10 GUI_TextSetMove

函数原型	rk_err_t GUI_TextSetMove(HGC pGc, int speed)
pGc	控件句柄
speed	延时参数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	启动文字走马灯效果。

5.5.11 GUI_TextTimer

函数原型	rk_err_t GUI_TextTimer(HGC pGc)
------	---------------------------------

pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	UI 定时器回调处理，用于实现跑马灯效果。

5.5.12 GUI_TextExtractString

函数原型	Ucs2 * GUI_TextExtractString(int resource)
resource	文字 SID
返回值	Unicode 字符串
功能说明	根据 SID 从固件中取得相应的字符串。

5.5.13 GUI_TextExtractString

函数原型	rk_err_t GUI_TextAutoAlign(FONT_TYPE font, int align, int x0, int y0, int xSize, int ySize, int * StartX, int * StartY, int * DestXSize, int * DestYSize, Ucs2 * text)
font	字体: TYPE12X12: 12*12 TYPE16X16: 16*16
align	对齐方式: TEXT_ALIGN_X_LEFT: X 轴左对齐 TEXT_ALIGN_X_RIGHT: X 轴右对齐 TEXT_ALIGN_X_CENTER: X 轴居中对齐 TEXT_ALIGN_Y_UP: Y 轴顶部对齐 TEXT_ALIGN_Y_DOWN: Y 轴底部对齐 TEXT_ALIGN_Y_CENTER: Y 轴居中对齐
x0	显示位置
y0	显示位置
xSize	文本框宽度

ySize	文本框高度
StartX	用于返回得出的起始位置
StartY	用于返回得出的起始位置
DestXSize	用于返回得出的文字宽度
DestYSize	用于返回得出的文字高度
text	Unicode 字符串
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据对齐方式及文本框大小，计算显示起始位置。

5.5.14 GUI_TextSizeCmp

函数原型	rk_err_t GUI_TextSizeCmp(FONT_TYPE font, Ucs2 * text, int size)
font	字体: TYPE12X12: 12*12 TYPE16X16: 16*16
text	Unicode 字符串
size	宽度
返回值	RK_SUCCESS: 文字长度 <= size RK_ERROR: 文字长度 > size
功能说明	比较字符串宽度是否超过 size，超过则返回 RK_ERROR。

5.5.15 GUI_TextExtractSize

函数原型	int GUI_TextExtractSize(FONT_TYPE font, Ucs2 * text)
font	字体: TYPE12X12: 12*12 TYPE16X16: 16*16
text	Unicode 字符串
返回值	文字宽度

功能说明	计算字符串宽度
------	---------

5.5.16 Font16_CompData / Font12_CompData

函数原型	uint16 Font16_CompData(uint8 * pBuf, unsigned short dat) uint16 Font12_CompData(uint8 * pBuf, unsigned short dat)
pBuf	数据缓冲，用于存放取得的点阵信息
dat	Unicode 字符
返回值	iType 标识信息： > 0x80: iType- 0x80 字符宽度（像素） = 0x80: 空格 = 0xFFFF: 错误
功能说明	从固件中取得对应字符的信息。

5.5.17 Font12_CompType / Font16_CompType

函数原型	uint16 Font12_CompType(unsigned short dat) uint16 Font16_CompType(unsigned short dat)
dat	Unicode 字符
返回值	iType 标识信息： > 0x80: iType- 0x80 字符宽度（像素） = 0x80: 空格 = 0xFFFF: 错误
功能说明	从固件中取得对应字符的 iType 信息。

6 MSGBOX 控件

6.1 MSGBOX 控件简介

MSGBOX，即消息管理控件。消息管理控件分为警告模式和对话模式两种。警告模式下，焦

点自动落在消息管理控件上，并拦截所有的输入事件（如按键按下）并透传给 APP。对话框模式下，焦点自动落在消息管理控件上。对于 FFD 短按、FFD 长按、FFW 短按、FFW 长按、MENU 短按等事件，控件将拦截接收，并根据具体事件更新 UI。无关按键和用户的输入信息，将通过回调函数通知 APP。

6.2 MSGBOX 创建

函数原型	HGC GuiTask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_MSG_BOX（创建 MSGBOX 控件时）
arg	RKGUI_MSGBOX_ARG 结构（创建 MSGBOX 控件时）
返回值	控件句柄
功能说明	MSGBOX 控件创建前，通常需要通过 GuiTask_AppReciveMsg 函数先行注册 APP 回调函数，用于接收用户输入和处理无关按键。

6.2.1 RKGUI_MSGBOX_ARG

```
typedef struct
{
    int display;
    int level;
    int x;
    int y;
    eMSGBOX_DISPLAY_CMD cmd;

    int title;
    int Button_On;
    int Button_Off;

    eTEXT_DISPLAY_CMD text_cmd;
    int align;
    void * text;
}RKGUI_MSGBOX_ARG;
```

名称	RKGUI_MSGBOX_ARG
display	隐藏标志，0 隐藏，1 显示。
level	图层，0 底层，1 顶层（即状态栏）。
x	显示位置

y	显示位置
cmd	创建模式 MSGBOX_CMD_DIALOG: 对话框模式 MSGBOX_CMD_WARNING: 警告框模式
title	标题文字 SID
Button_On	on 按钮文字 SID (仅对话框模式有效)
Button_Off	off 按钮文字 SID (仅对话框模式有效)
text_cmd	内容文字模式: TEXT_CMD_BUF: 普通模式, 同 TEXT TEXT_CMD_ID: SID 模式, 同 TEXT
align	内容文字对齐方式: TEXT_ALIGN_X_LEFT: X 轴左对齐 TEXT_ALIGN_X_RIGHT: X 轴右对齐 TEXT_ALIGN_X_CENTER: X 轴居中对齐 TEXT_ALIGN_Y_UP: Y 轴顶部对齐 TEXT_ALIGN_Y_DOWN: Y 轴底部对齐 TEXT_ALIGN_Y_CENTER: Y 轴居中对齐
text	内容文字: text_cmd= TEXT_CMD_BUF: unicode 字符串 text_cmd= TEXT_CMD_ID: 文字 SID

6.2.2 警告模式

```

RKGUI_MSGBOX_ARG pstMsgBoxArg;
    GuiTask_AppReciveMsg(GuiTask_MsgBoxCallBack);    // 注册 APP 回调
    pstMsgBoxArg.cmd= MSGBOX_CMD_WARNING;          // 警告模式
    pstMsgBoxArg.x= 4;                                // 显示位置 x,y(4,20)
    pstMsgBoxArg.y= 20;
    pstMsgBoxArg.display = 1;                          // 创建即显示
    pstMsgBoxArg.level= 0;                             // 图层 0
    pstMsgBoxArg.title= SID_WARNING;                   // 标题 SID
    pstMsgBoxArg.text_cmd= TEXT_CMD_ID;                // 内容文字采用 SID 模式
    pstMsgBoxArg.text= (void *)0;                      // 内容 SID
    
```



```
pstMsgBoxArg.align= TEXT_ALIGN_Y_CENTER|TEXT_ALIGN_X_CENTER;// 居中对齐
hGc= GUITask_CreateWidget(GUI_CLASS_MSG_BOX, &pstMsgBoxArg);
```

6.2.3 对话框模式

```
RKGUI_MSGBOX_ARG pstMsgBoxArg;
GuiTask_AppReciveMsg(GuiTask_MsgBoxCallBack);
pstMsgBoxArg.cmd= MSGBOX_CMD_DIALOG;
pstMsgBoxArg.x= 4;
pstMsgBoxArg.y= 20;
pstMsgBoxArg.display = 1;
pstMsgBoxArg.level= 0;
pstMsgBoxArg.title= SID_WARNING;
pstMsgBoxArg.text_cmd= TEXT_CMD_ID;
pstMsgBoxArg.text= (void *)0;
pstMsgBoxArg.Button_On= SID_YES;
pstMsgBoxArg.Button_Off= SID_NO;
pstMsgBoxArg.align= TEXT_ALIGN_Y_CENTER|TEXT_ALIGN_X_CENTER;
hGc= GUITask_CreateWidget(GUI_CLASS_MSG_BOX, &pstMsgBoxArg);
```

6.3 MSBOX 回调处理

函数原型	rk_err_t P_APP_RECVIE_MSG(APP_RECVIE_MSG_EVENT evnet_type, int evnet, void * arg, HGC pGc)
evnet_type	事件类型： APP_RECVIE_MSG_EVENT_KEY 普通按键 APP_RECVIE_MSG_EVENT_WARING 警告模式 APP_RECVIE_MSG_EVENT_DIALOG 对话框模式
evnet	事件子类型，具体意义与事件类型相关
arg	传入参数，具体意义与事件类型相关
pGc	控件句柄，用于标识产生回调的具体控件。如果不是由控件发起的回调，此值为 NULL。
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	注意：当回调函数发生时，程序处于 GUI 线程中。此时，严禁 APP 再次请

	求操作 GUI。否则将有可能造成重入，导致线程锁死。建议的做法是通过 OS 提供的消息队列机制，将事件发送到 APP 线程中处理。
--	---

6.3.1 警告模式回调

函数原型	rk_err_t P_APP_RECVIE_MSG(APP_RECVIE_MSG_EVENT evnet_type, int evnet, void * arg, HGC pGc)
evnet_type	APP_RECVIE_MSG_EVENT_WARING（警告模式）
evnet	事件子类型（详见 KeyDevice.h）
arg	警告模式为 NULL
pGc	控件句柄.
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

```
rk_err_t GuiTask_MsgBoxCallBack(uint32 EVENT_type, uint32 event, void * arg, HGC pGc)
{
    if(EVENT_type==APP_RECVIE_MSG_EVENT_KEY)
    {
        AskQueueArg.event = event;
        AskQueueArg.event_type = event_type;
        rkos_queue_send(AskQueue, &AskQueueArg, 0);
    }
}
```

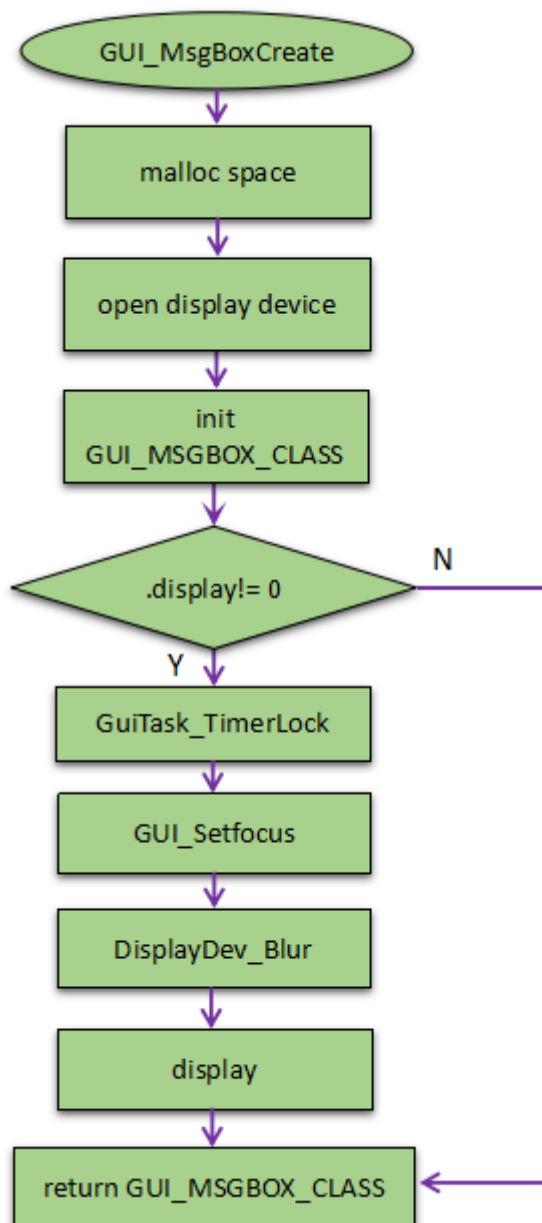
6.3.2 对话模式回调

函数原型	rk_err_t * P_APP_RECVIE_MSG(APP_RECVIE_MSG_EVENT evnet_type, int evnet, void * arg, HGC pGc)
evnet_type	APP_RECVIE_MSG_EVENT_DIALOG（警告模式）
evnet	事件子类型（详见 KeyDevice.h）
arg	警告模式为 NULL
pGc	控件句柄.
返回值	RK_SUCCESS: 执行成功

	RK_ERROR: 执行错误
--	----------------

```
rk_err_t GuiTask_MsgBoxCallBack(uint32 EVENT_type, uint32 event, void * arg, HGC pGc)
{
    if(EVENT_type==APP_RECIVE_MSG_EVENT_DIALOG)
    {
        if(arg){
            printf("用户取消\n");
        }
        else{
            printf("用户确定\n");
        }
    }
    else if(EVENT_type==APP_RECIVE_MSG_EVENT_KEY)
    {
        printf("其他按键:%x\n",event);
    }
}
```

6.4 MSBOX 创建流程



MSGBOX 控件，创建即是焦点除其他控件的内存分配和参数初始化外，显示控件时，还需要通过 `GuiTask_TimerLock` 来暂 UI 定时器，通过 `GUI_Setfocus` 来创建焦点，最后通过 `DisplayDev_Blur` 函数对整个 LCD 进行半透明显示，最后显示控件。

```
COMMON API GUI_CONTROL_CLASS * GUI_MsgBoxCreate(void *arg)
{
    PICTURE_INFO_STRUCT psPictureInfo;
    GUI_MSGBOX_CLASS * pstClass;
    RKGUI_MSGBOX_ARG * pstMsgBoxArg= (RKGUI_MSGBOX_ARG *)arg;

    pstClass= rkos_memory_malloc(sizeof(GUI_MSGBOX_CLASS));
    if (pstClass== NULL)
    {
        printf("GUI_MsgBoxCreate: malloc GUI_MSGBOX_CLASS fault\n");
        return NULL;
    }

    pstClass->pfControl.hDisplay= RKDev_Open(DEV_CLASS_DISPLAY, pstMsgBoxArg->level, NULL);
    if (pstClass->pfControl.hDisplay== NULL)
    {
        printf("GUI_MsgBoxCreate: open display fault\n");
        rkos_memory_free(pstClass);
        return NULL;
    }

    pstClass->pfControl.ClassID = GUI_CLASS_MSG_BOX;
    pstClass->pfControl.x0 = pstMsgBoxArg->x;
    pstClass->pfControl.y0 = pstMsgBoxArg->y;
    pstClass->pfControl.display= pstMsgBoxArg->display;
    pstClass->pfControl.level= pstMsgBoxArg->level;
    pstClass->pfControl.rotate= ROTATE_NOMAL;

    pstClass->cmd= pstMsgBoxArg->cmd;
    pstClass->text_cmd= pstMsgBoxArg->text_cmd;

    if ( pstMsgBoxArg->text_cmd==TEXT_CMD_ID )
    {
        pstClass->text= GUI_TextExtractString((int)pstMsgBoxArg->text);
        if ( pstClass->text== NULL )
        {
            return NULL;
        }
    }
    else
    {
        pstClass->text= pstMsgBoxArg->text;
    }

    pstClass->title= pstMsgBoxArg->title;
    pstClass->Button_On= pstMsgBoxArg->Button_On;
    pstClass->Button_Off= pstMsgBoxArg->Button_Off;
    pstClass->on_off= 0;
    pstClass->pfControl.flag= 0;
    pstClass->pfControl.lucency= OPACITY;
    pstClass->pfControl.translucence= 1;
    pstClass->align= pstMsgBoxArg->align;

    if( pstClass->cmd==MSGBOX_CMD_WARNING )
    {
        pstClass->Backdrop= IMG_ID_WARNING_BOX;
    }
    else if( pstClass->cmd==MSGBOX_CMD_DIALOG )
    {
        pstClass->Backdrop= IMG_ID_DIALOGBOX_BACKGROUND;
    }
}
```

```
else
{
    return NULL;
}

if( FW_GetPicInfoWithIDNum(pstClass->Backdrop, &psPictureInfo) != RK_SUCCESS )
{
    return NULL;
}
pstClass->pfControl.xSize = psPictureInfo.xSize;
pstClass->pfControl.ySize = psPictureInfo.ySize;

if(pstClass->pfControl.display)
{
    GuiTask_TimerLock();
    GUI_Setfocus(pstClass);
    DisplayDev_Blur(pstClass->pfControl.hDisplay);
    GUI_MsgBoxDisplay(pstClass);
    GUI_Createfocus(GUI_MsgBoxCallBack, pstClass);
}

return &pstClass->pfControl;
}
```

6.5 实现说明

6.5.1 GUI_MSGBOX_CLASS

```
typedef struct _GUI_MSGBOX_CLASS
{
    GUI_CONTROL_CLASS pfControl;
    eMSGBOX_DISPLAY_CMD cmd;
    int on_off;

    int title;
    int Button_On;
    int Button_Off;

    int Backdrop;

    eTEXT_DISPLAY_CMD text_cmd;
    int align;
    void * text;
}GUI_MSGBOX_CLASS;
```

名称	GUI_MSGBOX_CLASS
pfControl	控件类，包含了控件的基本信息。
cmd	模式选择。 MSGBOX_CMD_DIALOG：对话框模式 MSGBOX_CMD_WARNING：警告框模式
on_off	当前光标状态。0 确定 1 取消（仅对话框模式有效）

Button_On	On 按键文字 SID
Button_Off	Off 按键文字 SID
Backdrop	背景图片 SID
text_cmd	内容文字模式： TEXT_CMD_BUF: 普通模式 TEXT_CMD_ID: SID 模式
align	内容文字对齐方式： TEXT_ALIGN_X_LEFT: X 轴左对齐 TEXT_ALIGN_X_RIGHT: X 轴右对齐 TEXT_ALIGN_X_CENTER: X 轴居中对齐 TEXT_ALIGN_Y_UP: Y 轴顶部对齐 TEXT_ALIGN_Y_DOWN: Y 轴底部对齐 TEXT_ALIGN_Y_CENTER: Y 轴居中对齐
text	内容文字

6.5.2 GUI_MsgBoxCreate

函数原型	GUI_CONTROL_CLASS * GUI_MsgBoxCreate(void *arg)
arg	创建参数, RKGUI_MSGBOX_ARG 类型
返回值	NULL: 创建失败 非 NULL: 控件句柄
功能说明	根据创建参数, 创建控件, 并返回控件句柄。

6.5.3 GUI_MsgBoxDelete

函数原型	rk_err_t GUI_MsgBoxDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据句柄回收控件资源。

6.5.4 GUI_MsgBoxDisplay

函数原型	rk_err_t GUI_MsgBoxDisplay(HDC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示控件。

6.5.5 GUI_MsgBoxCallBack

函数原型	rk_err_t GUI_MsgBoxCallBack(uint32 evnet_type, uint32 event, void * arg, HGC pGc)
evnet_type	事件类型: GUI_EVENT_KEY: 按键事件
event	事件子类型
arg	带入参数
pGc	控件句柄, 用于标识引起回调的控件。
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	MSGBOX 创建即是焦点, 此函数负责所有输入事件的处理。

7 PROCESSBAR 控件

7.1 PROCESSBAR 控件简介

PROCESSBAR, 即进度条控件。此控件能够高效的管理 ICON 的比例缩放。适用于音量条、音乐进度条等长宽变化频繁的图标显示。

7.2 PROCESSBAR 创建

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_PROGRESSBAR（创建 PROCESSBAR 控件时）
arg	RKGUI_PROGRESSBAR_ARG 结构（创建 PROCESSBAR 控件时）
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误

7.2.1 RKGUI_PROGRESSBAR_ARG

```
typedef struct
{
    int display;
    int x;
    int y;
    int level;

    int Backdrop;
    int BackdropX;
    int BackdropY;

    int Bar;
    int percentage;

    ePROGRESSBAR_OPERATE_CMD cmd;
}RKGUI_PROGRESSBAR_ARG;
```

名称	RKGUI_PROGRESSBAR_ARG
display	隐藏标志，0 隐藏，1 显示。
x	显示位置
y	显示位置
level	图层，0 底层，1 顶层（即状态栏）。
Backdrop	背景图片 SID。
BackdropX	背景偏移
BackdropY	背景偏移
Bar	进度条图片 SID
percentage	宽度控制，范围 0~10000。55.55% = 5555

cmd	操作模式选择： PROGRESSBAR_SET_CONTENT：修改宽度
-----	---

7.2.2 参考例程

```
HGC hGc;
RKGUI_PROGRESSBAR_ARG pstProgressArg;
pstProgressArg.x= 7;                // 显示位置 x,y(7,88)
pstProgressArg.y= 88;
pstProgressArg.display = 1;          // 创建即显示
pstProgressArg.level= 0;             // 图层 0

pstProgressArg.Backdrop= IMG_ID_MUSIC_BACKGROUND;    // 背景 SID
pstProgressArg.BackdropX= 7;          // 背景偏移 x,y(7,78)
pstProgressArg.BackdropY= 78;

pstProgressArg.Bar= IMG_ID_PROGRESS_BAR;    // 进度条 SID 编号
pstProgressArg.percentage= 10000;          // 100%宽度
hGc= GUITask_CreateWidget(GUI_CLASS_PROGRESSBAR, &pstProgressArg);
```

7.3 PROGRESSBAR 内容变更

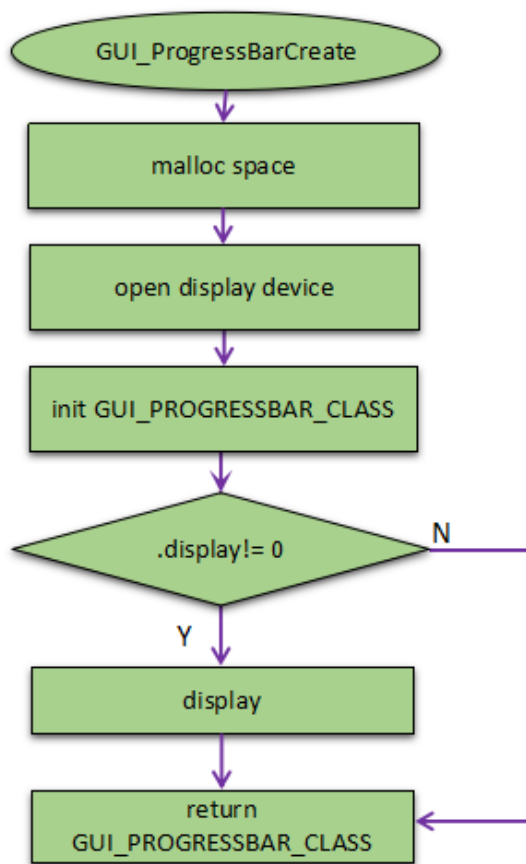
函数原型	rk_err_t GuiTask_OperWidget(HGC pGc, eRKGUI_OPERATE_CMD cmd, void *arg, uint32 mode)
pGc	控件句柄
cmd	OPERATE_SET_CONTENT。（替换内容时）
arg	RKGUI_PROGRESSBAR_ARG 具体意义与命令参数有关。
mode	SYNC_MODE：同步模式 ASYNC_MODE：异步模式
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误

7.3.1 长度变更

```
RKGUI_PROGRESSBAR_ARG pstProgressBarContent;
pstProgressBarContent.cmd= PROGRESSBAR_SET_CONTENT;
```

```
pstProgressBarContent.percentage= 5000; // 50.00%  
GuiTask_OperWidget(hGc, OPERATE_SET_CONTENT, &pstProgressBarContent,  
SYNC_MODE);
```

7.4 PROGRESSBAR 创建流程



PROGRESSBAR 控件的创建过程与其他控件基本一致。值得一提的是：为了加快处理速度，减少访问固件资源次数。PROGRESSBAR 在创建时，将会通过 `rkos_memory_malloc` 分配内存空间，并将整个图片资源读入内存，然后显示。这部分内存控件，将一直保持，直到控件被删除为止。

```
COMMON API GUI_CONTROL_CLASS * GUI_ProgressBarCreate(void *arg)
{
    GUI_PROGRESSBAR_CLASS * pstProgressBarClass;
    RKGUI_PROGRESSBAR_ARG * pstProgressBarArg= (RKGUI_PROGRESSBAR_ARG *)arg;;
    PICTURE_INFO_STRUCT psPictureInfo;

    pstProgressBarClass= rkos_memory_malloc(sizeof(GUI_PROGRESSBAR_CLASS));
    if (pstProgressBarClass== NULL)
    {
        return NULL;
    }

    pstProgressBarClass->pfControl.hDisplay= RKDev_Open(DEV_CLASS_DISPLAY, pstProgressBarArg->level, NULL);
    if (pstProgressBarClass->pfControl.hDisplay== NULL)
    {
        printf("Create ProgressBar: open display fault\n");
        rkos_memory_free(pstProgressBarClass);
        return NULL;
    }

    pstProgressBarClass->pfControl.ClassID = GUI_CLASS_PROGRESSBAR;
    pstProgressBarClass->pfControl.x0 = pstProgressBarArg->x;
    pstProgressBarClass->pfControl.y0 = pstProgressBarArg->y;
    pstProgressBarClass->pfControl.rotate= ROTATE_NOMAL;
    pstProgressBarClass->pfControl.level= pstProgressBarArg->level;

    pstProgressBarClass->pfControl.display= pstProgressBarArg->display;
    pstProgressBarClass->pfControl.flag= 0;

    pstProgressBarClass->pfControl.lucency= OPACITY;
    pstProgressBarClass->pfControl.translucence= 0;

    pstProgressBarClass->Backdrop = pstProgressBarArg->Backdrop;
    pstProgressBarClass->BackdropX= pstProgressBarArg->BackdropX;
    pstProgressBarClass->BackdropY= pstProgressBarArg->BackdropY;
    pstProgressBarClass->Bar= pstProgressBarArg->Bar;
    pstProgressBarClass->percentage= pstProgressBarArg->percentage;

    if( pstProgressBarClass->pfControl.display )
    {
        if( FW_GetPicInfoWithIDNum(pstProgressBarClass->Bar, &psPictureInfo) != RK_SUCCESS )
        {
            return NULL;
        }

        pstProgressBarClass->dat= (uint8 *)rkos_memory_malloc( psPictureInfo.totalSize);
        if(FW_GetPicResource(psPictureInfo.offsetAddr, pstProgressBarClass->dat, psPictureInfo.totalSize) != RK_SUCCESS)
        {
            RKDev_Close(pstProgressBarClass->pfControl.hDisplay);
            rkos_memory_free(pstProgressBarClass->dat);
            rkos_memory_free(pstProgressBarClass);
            return NULL;
        }

        pstProgressBarClass->BarX = psPictureInfo.xSize;
        pstProgressBarClass->BarY = psPictureInfo.ySize;

        if(pstProgressBarClass->percentage== 0)
        {
            pstProgressBarClass->pfControl.xSize = 0;
        }
        else
        {
            pstProgressBarClass->pfControl.xSize = (pstProgressBarClass->BarX* pstProgressBarClass->percentage)/ 10000;
        }
        pstProgressBarClass->pfControl.ySize = pstProgressBarClass->BarY;

        GUI_ProgressBarDisplay(pstProgressBarClass);
    }

    return &pstProgressBarClass->pfControl;
}
```

7.5 实现说明

7.5.1 GUI_PROGRESSBAR_CLASS

```
typedef struct _GUI_PROGRESSBAR_CLASS
{
    GUI_CONTROL_CLASS pfControl;

    int Backdrop;
    int BackdropX;
    int BackdropY;

    int Bar;
    int BarX;
    int BarY;
    int percentage;
    uint8 * dat;
}GUI_PROGRESSBAR_CLASS;
```

名称	GUI_PROGRESSBAR_CLASS
pfControl	控件类，包含了控件的基本信息。
Backdrop	背景图片 SID。
BackdropX	背景偏移
BackdropY	背景偏移
Bar	进度条图片 SID
BarX	进度条总长
BarY	进度条总高
percentage	当前进度（百分比表示）
dat	dat 图片点阵数据

7.5.2 GUI_ProgressBarCreate

函数原型	GUI_CONTROL_CLASS * GUI_ProgressBarCreate(void *arg)
arg	创建参数，RKGUI_PROGRESSBAR_ARG 类型
返回值	NULL：创建失败 非 NULL：控件句柄

功能说明	根据创建参数，创建控件，并返回控件句柄。
------	----------------------

7.5.3 GUI_ProgressBarDelete

函数原型	rk_err_t GUI_ProgressBarDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	根据句柄回收控件资源。

7.5.4 GUI_ProgressBarDisplay

函数原型	rk_err_t GUI_ProgressBarDisplay(HDC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	显示控件。

7.5.5 GUI_ProgressBarContent

函数原型	rk_err_t GUI_ProgressBarContent(HGC pGc, void * arg)
pGc	控件句柄
arg	带入参数，RKGUI_PROGRESSBAR_ARG 类型
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	用于修改控件显示内容： PROGRESSBAR_SET_CONTENT: 控件长度 PROGRESSBAR 此控件始终为非透明模式，修改控件内容，只会刷新当前控件。不会出发遮叠判断。

8 CHAIN 控件

8.1 CHAIN 控件简介

CHAIN 控件，即图片连播控件。此控件能够高效的管理一组图标的连续播放。

8.2 CHAIN 创建

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_CHAIN（创建 CHAIN 控件时）
arg	RKGUI_CHAIN_ARG 结构（创建 CHAIN 控件时）
返回值	控件句柄

8.2.1 RKGUI_CHAIN_ARG

```
typedef struct
{
    int display;
    uint32 resource;
    int num;
    int x;
    int y;
    int delay;
    int level;
    int blurry;
}RKGUI_CHAIN_ARG;
```

名称	RKGUI_CHAIN_ARG
display	隐藏标志，0 隐藏，1 显示。
resource	连播起始图片 SID。
num	连播图片数量
x	显示位置
y	显示位置
delay	延迟控制
level	图层，0 底层，1 顶层（即状态栏）。

blurry

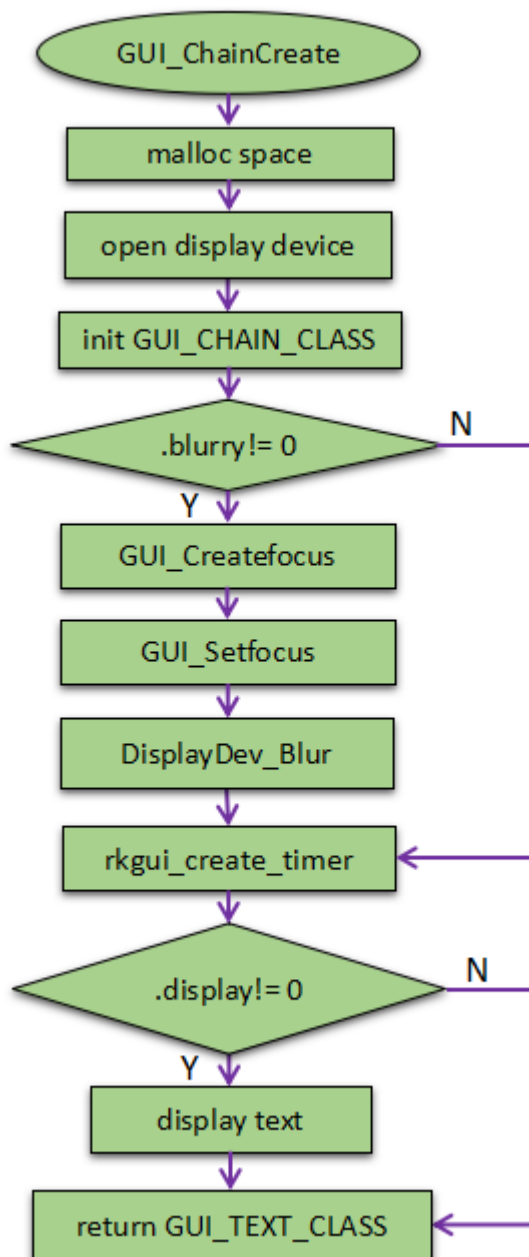
虚化背景控制。0 不虚化背景 1 虚化背景，并自动设为焦点

8.2.2 示例代码

```
HGC hGc;
RKGUI_CHAIN_ARG pstChainArg;

pstChainArg.x= 20;           // 显示位置 x,y(20,40)
pstChainArg.y= 40;
pstChainArg.resource = IMG_ID_WAIT0;    // 图片起始编号 SID
pstChainArg.num = 16;        // 连续播放 16 张
pstChainArg.delay = 10;      // 延时 10 个节拍
pstChainArg.level = 0;       // 图层 0
pstChainArg.display = 1;     // 创建即显示
pstChainArg.blurry = 1;      // 虚化背景
hGc= GUITask_CreateWidget(GUI_CLASS_CHAIN, &pstChainArg);
```


8.3 CHAIN 创建流程



CHAIN 控件创建的基本步骤与其他控件基本一致。值得注意的是：CHAIN 控件创建时用户可以通过 `blurry` 参数来决定 CHAIN 控件创建时是否需要虚化背景。虚化背景的情况下，CHAIN 控件创建即是焦点，但控件通常不处理任何事件，而是利用焦点系统拦截按键信息，防止其他控件或应用程序接收到输入事件后操作 UI 造成混乱。

```
COMMON API GUI_CONTROL_CLASS * GUI_ChainCreate(void *arg)
{
    PICTURE_INFO_STRUCT psPictureInfo;
    GUI_CHAIN_CLASS * pstClass;
    RKGUI_CHAIN_ARG * pstChainArg= (RKGUI_CHAIN_ARG *)arg;

    pstClass= rkos_memory_malloc(sizeof(GUI_CHAIN_CLASS));
    if (pstClass== NULL)
    {
        printf("GUI_ChainCreate: malloc GUI_CHAIN_CLASS fault\n");
        return NULL;
    }

    pstClass->pfControl.hDisplay= RKDev_Open(DEV_CLASS_DISPLAY, pstChainArg->level, NULL);
    if (pstClass->pfControl.hDisplay== NULL)
    {
        printf("GUI_ChainCreate: open display fault\n");
        rkos_memory_free(pstClass);
        return NULL;
    }

    pstClass->pfControl.ClassID = GUI_CLASS_CHAIN;
    pstClass->pfControl.x0 = pstChainArg->x;
    pstClass->pfControl.y0 = pstChainArg->y;
    pstClass->pfControl.xSize = -1;
    pstClass->pfControl.ySize = -1;

    pstClass->pfControl.display= pstChainArg->display;
    pstClass->pfControl.level= pstChainArg->level;

    pstClass->pfControl.rotate= ROTATE_NOMAL;
    pstClass->ResourceId= pstChainArg->resource;
    pstClass->Current= pstChainArg->resource;

    pstClass->ResourceId= pstChainArg->resource;
    pstClass->Current= pstChainArg->resource;
    pstClass->num= pstChainArg->num;
    pstClass->delay= pstChainArg->delay;
    pstClass->speed= 0;

    if( FW_GetPicInfoWithIDNum(pstClass->Current, &psPictureInfo)!= RK_SUCCESS )
    {
        //return (GUI_CONTROL_CLASS *) RK_ERROR;
        return &pstClass->pfControl;
    }

    if(pstClass->pfControl.x0< 0)pstClass->pfControl.x0= psPictureInfo.x;
    if(pstClass->pfControl.y0< 0)pstClass->pfControl.y0= psPictureInfo.y;
    if(pstClass->pfControl.xSize< 0)pstClass->pfControl.xSize = psPictureInfo.xSize;
    if(pstClass->pfControl.ySize< 0)pstClass->pfControl.ySize = psPictureInfo.ySize;

    pstClass->blurry = pstChainArg->blurry;
    if(pstChainArg->blurry)
    {
        GUI_Createfocus(GUI_ChainCallBack, pstClass);
        GUI_Setfocus(pstClass);
        DisplayDev_Blur(pstClass->pfControl.hDisplay);
    }

    rkgui_create_timer(pstClass,GUI_ChainTimer);
}
```

```
if(pstChainArg->display)
{
    rkgui_start_timer(pstClass);
    GUI_ChainDisplay(pstClass);
}

pstClass->pfControl.translucence= pstChainArg->blurry;
pstClass->pfControl.lucency= OPACITY;
pstClass->pfControl.flag = 0;

return &pstClass->pfControl;
}
```

8.4 实现说明

8.4.1 GUI_CHAIN_CLASS

```
typedef struct _GUI_CHAIN_CLASS
{
    GUI_CONTROL_CLASS pfControl;
    uint32 ResourceId;
    uint32 Current;
    int num;
    int delay;
    int speed;
    int blurry;
}GUI_CHAIN_CLASS;
```

名称	GUI_CHAIN_CLASS
pfControl	控件类，包含了控件的基本信息。
ResourceId	连播起始图片 SID
Current	当前图片 SID
num	连播图片数量
delay	延时参数
speed	连播速度
blurry	背景虚化标志。 0 不虚化背景 1 虚化背景

8.4.2 GUI_ChainCreate

函数原型	GUI_CONTROL_CLASS * GUI_ChainCreate(void *arg)
------	--

arg	创建参数，RKGUI_CHAIN_ARG 类型
返回值	NULL：创建失败 非 NULL：控件句柄
功能说明	根据创建参数，创建控件，并返回控件句柄。

8.4.3 GUI_ChainDelete

函数原型	rk_err_t GUI_ChainDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	根据句柄回收控件资源。

8.4.4 GUI_ChainDisplay

函数原型	rk_err_t GUI_ChainDisplay(HDC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	显示控件。

9 SELECT 控件

9.1 SELECT 控件简介

SELECT 控件，即表单控件。使用 SELECT 控件能够高效、便捷的生成和管理表单数据。

9.2 SELECT 创建

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_SELECT（创建 SELECT 控件时）
arg	RKGUI_SELECT_ARG 结构（创建 SELECT 控件时）
返回值	控件句柄
功能说明	SELECT 控件创建即是焦点，并且将拦截部分按键。对于无法处理的事件信息，将传递给 APP。

9.2.1 RKGUI_SELECT_ARG

```
typedef rk_err_t (* P_SELECT_RECVIE_MSG)(HGC pGc, eSELECT_EVENT_TYPE evnet_type, void * item, int offset);
typedef struct
{
    int x;
    int y;
    int xSize;
    int ySize;
    int level;
    int display;

    int itemNum;
    int Cursor;
    int CursorOffset;
    int Background;
    int MaxDisplayItem;

    int IconBoxSize;
    int SeekBar;
    int CursorStyle;
    P_SELECT_RECVIE_MSG pReviceMsg;
};RKGUI_SELECT_ARG;
```

名称	RKGUI_SELECT_ARG
x	显示位置
y	显示位置
xSize	控件宽度
ySize	控件高度
level	图层，0 底层，1 顶层（即状态栏）。
display	隐藏标志，0 隐藏，1 显示。
itemNum	单元总数
Cursor	光标位置

CursorOffset	光标偏移
Background	背景图片 SID
MaxDisplayItem	表格能够显示的最大单元数
IconBoxSize	图标区大小
SeekBar	拖动条 SID
CursorStyle	光标图片 SID
pReviceMsg	回调函数

9.2.2 示例代码

```
HGC pGc;
RKGUI_SELECT_ARG stSelectArg;

stSelectArg.x= 0;           // 显示位置 x,y(0,20)
stSelectArg.y= 20;
stSelectArg.xSize= 128;     // 表单大小
stSelectArg.ySize= 137;
stSelectArg.display = 1;    // 创建即显示
stSelectArg.level= 0;      // 图层 0
stSelectArg.Background= IMG_ID_BROWSER_BACKGROUND; // 表单背景编号
stSelectArg.SeekBar= IMG_ID_BROWSER_SCOLL2;        // 拖动条背景编号
stSelectArg.Cursor= 1;      // 创建时光标位置
stSelectArg.CursorOffset= 0; // 光标偏移
stSelectArg.itemNum= 14;     // 单元格总数
stSelectArg.MaxDisplayItem= 8; // 最大能够显示单元格数量
stSelectArg.IconBoxSize= 18; // 图标显示区大小

stSelectArg.pReviceMsg= GuiTask_SelectCallBack; // 回调函数
stSelectArg.CursorStyle= IMG_ID_SEL_ICON;      // 光标编号 SID
pGc= GUITask_CreateWidget(GUI_CLASS_SELECT, &stSelectArg);
```

9.3 SELECT 回调处理

函数原型	rk_err_t P_SELECT_RECIVE_MSG(HGC pGc, eSELECT_EVENT_TYPE evnet_type, RKGUI_SELECT_ITEM* item, int offset)
pGc	控件句柄，用于标识发生回调的具体控件

evnet_type	事件类型： SELECT_ENVEN_ENTER: 用户确认事件 SELECT_ENVEN_UPDATA: 请求数据事件
item	数据指针，SELECT_ENVEN_ENTER 模式时始终为 NULL
offset	偏移标识： SELECT_ENVEN_ENTER: 用户选中第几个单元格 SELECT_ENVEN_UPDATA: 本次请求的是第几个单元格的数据
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	当控件管理器接收到 APP 发来的创建 SELECT 控件请求或表单发生翻页行为时，SELECT 将会根据创建时传入的回调函数向 APP 请求内容。

```
typedef struct _RKGUI_SELECT_ITEM
{
    eTEXT_DISPLAY_CMD cmd;
    Ucs2 * text;
    int text_id;
    int sel_icon;
    int unsel_icon;
}RKGUI_SELECT_ITEM;
```

名称	RKGUI_SELECT_ITEM
cmd	文字模式选择： TEXT_CMD_BUF TEXT_CMD_ID
text	Unicode 字符串（仅 TEXT_CMD_BUF 模式有效）
text_id	文字 SID（仅 TEXT_CMD_ID 模式有效）
sel_icon	选中时，小图标 SID。< 0 无小图标
unsel_icon	非选中时，小图标 SID。< 0 无小图标

9.3.1 SID 模式

rk_err_t GuiTask_SelectCallBack(HGC pGc, eSELECT_EVENT_TYPE EVENT_type,

```
RKGUI_SELECT_ITEM * item, int offset)
{
    switch(EVENT_type)
    {
        case SELECT_ENVEN_UPDATA:
            item->cmd= TEXT_CMD_ID;
            item->text_id= offset;
            item->sel_icon= IMG_ID_SETMENU_ICON_SEL;
            item->unsel_icon= IMG_ID_SETMENU_ICON_NOSEL;
            break;
    }
    return RK_SUCCESS;
}
```

9.3.2 普通模式

```
rk_err_t GuiTask_SelectCallBack(HGC pGc, eSELECT_EVENT_TYPE EVENT_type,
RKGUI_SELECT_ITEM * item, int offset)
{
    switch(EVENT_type)
    {
        case SELECT_ENVEN_UPDATA:
            item->cmd= TEXT_CMD_BUF;
            memcpy(item->text, TEXT("SELECT 回调示例"), sizeof(TEXT("SELECT 回调示
例")) );
            item->sel_icon= IMG_ID_SETMENU_ICON_SEL;
            item->unsel_icon= IMG_ID_SETMENU_ICON_NOSEL;
            break;
    }
    return RK_SUCCESS;
}
```

注意：普通模式下，SELECT 控件内部将开辟空间暂存用户的字符串数据。

9.4 SELECT 内容变更

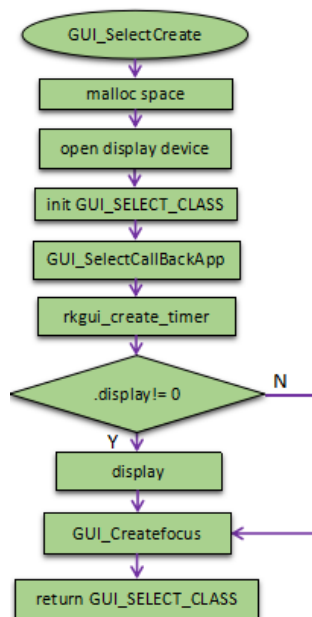
函数原型	rk_err_t GuiTask_OperWidget(HGC pGc, eRKGUI_OPERATE_CMD cmd, void *arg, uint32 mode)
pGc	控件句柄
cmd	OPERATE_SET_CONTENT。（替换内容时）

arg	RKGUI_SELECT_CONTENT 结构。（替换内容时）
mode	SYNC_MODE: 同步模式 ASYNC_MODE: 异步模式
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	当单元格总数发生变化时，控件能够自动判断当前状态下是否需要像 APP 请求新的数据，以刷新显示。

```

RKGUI_SELECT_CONTENT parameter;
parameter.itemNum= 19;           // 单元总数
GuiTask_OperWidget(pGc, OPERATE_SET_CONTENT, &parameter, SYNC_MODE);
    
```

9.5 SELECT 创建流程



SELECT 控件创建即是焦点，除了必要开启 Display Device 等基本步骤外，控件还会调用 GUI_Createfocus 和 GUI_Setfocus 函数创建和设置焦点。为避免记录整个表单显示内容需要耗费过多存储空间的问题，SELECT 控件仅在创建、光标翻页时才会通过 GUI_SelectCallBackApp 回调创建者申请填写表单内容。当用户按下确定按钮时，也将通过回调函数，告知创建者被按下的单元格偏移。

此外，值得注意的是：如果单元格使用了 TEXT_CMD_BUF 模式，控件将计算字符串长度，分配内存控件，并拷贝。直到控件删除或单元格移出显示区后，才会释放。

9.6 实现说明

9.6.1 GUI_SELECT_CLASS

```
typedef struct _GUI_SELECT_CLASS
{
    GUI_CONTROL_CLASS pfControl;
    eTEXT_DISPLAY_CMD cmd;

    RKGUI_SELECT_ITEM * pList;
    Ucs2 * text;
    Ucs2 * text_bck;
    int pListCurrent;
    int pDisplayCurrent;

    int Background;
    int SeekBar;
    int ItemHeight;
    int IconAreaStart;
    int IconAreaSize;

    int ItemWidth;
    int CursorStyle;
    int ItemTopMargin;

    uint32 itemSum;
    uint32 limit;
    uint32 MaxDisplayItem;

    int CursorOffset;
    int Cursor;

    P_SELECT_RECIVE_MSG pReviceMsg;

    int BarX;
    int BarY;
    int BarXSize;
    int BarYSize;
    int BarYSizeBck;
    float BarYScale;

    int KeepBackground;
}GUI_SELECT_CLASS;
```

名称	RKGUI_SELECT_ITEM
pfControl	控件类，包含了控件的基本信息。
cmd	文字模式选择： TEXT_CMD_BUF TEXT_CMD_ID
pList	显示单元缓冲区，用于缓存用户填入的单元格显示内容。
text	光标所在单元格，当前文字指针

text_bck	光标所在单元格，当前文字指针
pListCurrent	指向当前显示单元缓冲区。即指向光标对应单元格的显示内容。
pDisplayCurrent	指向显示单元缓冲区头部。
Background	背景图标 SID
SeekBar	拖动条图片 SID
ItemHeight	单元格高度
IconAreaStart	小图标区域起始地址
IconAreaSize	小图标区域大小
ItemWidth	单元格宽度
CursorStyle	光标 SID
ItemTopMargin	单元格顶部留边大小
itemSum	单元格总数
limit	总数限制： $\text{MaxDisplayItem} < \text{itemSum} = \text{MaxDisplayItem}$ $\text{MaxDisplayItem} \geq \text{itemSum} = \text{itemSum}$
MaxDisplayItem	表格的单元格总数，即最多能够显示多少个单元格
CursorOffset	光标总偏移，用于指明光标所在单元格位于总表的偏移值。
Cursor	光标偏移，用于指明光标位于第几个单元格。
pReviceMsg	回调函数
BarX	拖动条显示位置
BarY	拖动条显示位置
BarXSize	拖动条宽度
BarYSize	拖动条高度
BarYSizeBck	拖动条光标 SID
BarYScale	拖动条刻度
KeepBackground	文字滚动时，是否需要刷新背景。 1: 无需刷新背景 0: 刷新背景

9.6.2 GUI_SelectCreate

函数原型	GUI_CONTROL_CLASS * GUI_SelectCreate(void *arg)
arg	创建参数，RKGUI_SELECT_ARG 类型
返回值	NULL：创建失败 非 NULL：控件句柄
功能说明	根据创建参数，创建控件，并返回控件句柄。

9.6.3 GUI_SelectDelete

函数原型	rk_err_t GUI_SelectDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	根据句柄回收控件资源。

9.6.4 GUI_SelectDisplay

函数原型	rk_err_t GUI_SelectDisplay(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	显示控件。

9.6.5 DisplayMenuStrWithIDNum

函数原型	rk_err_t DisplayMenuStrWithIDNum(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误

功能说明	刷新一整张表。
------	---------

9.6.6 GUI_SelectSetContent

函数原型	rk_err_t GUI_SelectSetContent(HGC pGc ,RKGUI_SELECT_CONTENT * parameter)
pGc	控件句柄
parameter	带入参数, RKGUI_SELECT_CONTENT 类型
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	修改表单内容。

9.6.7 GUI_SelectCallBack

函数原型	rk_err_t GUI_SelectCallBack(uint32 evnet_type, uint32 event, void * arg, HGC pGc)
evnet_type	输入事件类型
event	输入事件子类型
arg	带入参数
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	输入事件回调处理。

9.6.8 GUI_SelectCallBackApp

函数原型	rk_err_t GUI_SelectCallBackApp(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

功能说明	回调创建者，请求显示内容。如果内容使用 TEXT_CMD_BUF 模式，将计算长度，分配空间，并拷贝。
------	---

9.6.9 GUI_SelectClearDrag

函数原型	rk_err_t GUI_SelectClearDrag(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	隐藏拖动条。

9.6.10 GUI_SelectDrag

函数原型	rk_err_t GUI_SelectDrag(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	绘制拖动条。

9.6.11 GUI_CellCursorDisplay

函数原型	rk_err_t GUI_CellCursorDisplay(HDC hDisplay, int x0, int y0, int Cursor, int ItemHeight, int IconAreaSize, int CursorStyle)
hDisplay	控件句柄
x0	整表显示位置
y0	整表显示位置
Cursor	光标偏移
ItemHeight	单元格高度
IconAreaSize	小图标区域宽度
CursorStyle	光标 SID 编号

返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	在指定位置显示光标。

9.6.12 GUI_ClearBackCursor

函数原型	rk_err_t GUI_ClearBackCursor(HDC hDisplay, int x0, int y0, int limit, int Cursor, int Background, int ItemHeight, int IconAreaSize, int ItemWidth, int ItemTopMargin)
hDisplay	Display 设备句柄
x0	整表显示位置
y0	整表显示位置
limit	实际能够显示的单元数
Cursor	光标偏移
Background	背景 SID
ItemHeight	单元格高度
IconAreaSize	小图标区域宽度
ItemWidth	单元格宽度
ItemTopMargin	表格顶部留边区域大小
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	隐藏光标显示。

9.6.13 GUI_ItemDisplay

函数原型	rk_err_t GUI_ItemDisplay(HGC pGc, int Cursor, RKGUI_SELECT_ITEM * item)
pGc	控件句柄
Cursor	光标偏移
item	显示内容

返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	在第 Cursor 个单元格内显示 item 的内容。

9.6.14 GUI_SelectStopTimer

函数原型	rk_err_t GUI_SelectStopTimer(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	停止文字走马灯效果。

9.6.15 GUI_SelectStartTimer

函数原型	rk_err_t GUI_SelectStartTimer(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	如果光标所在的文字长度超出单元格，则开启文字走马灯效果。

9.6.16 GUI_SelectStartTimer

函数原型	rk_err_t GUI_SelectTimer(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	UI 定时器回调函数，用于完成跑马灯效果。

10 SPECTRUM 控件

10.1 SPECTRUM 控件简介

SPECTRUM 控件，即频谱管理控件。该控件专为频谱显示设计，能够更为高效的显示和管理频谱图像的显示。

10.2 SPECTRUM 创建

函数原型	HGC GUITask_CreateWidget(eGc ClassID, void * arg)
ClassID	GUI_CLASS_SPECTRUM（创建 SPECTRUM 控件时）
arg	RKGUI_SPECTRUM_ARG 结构（创建 SPECTRUM 控件时）
返回值	控件句柄

10.2.1 RKGUI_SPECTRUM_ARG

```
typedef struct
{
    int display;
    int x;
    int y;
    int level;

    int StripStyle;
    int Backdrop;
    int BackdropX;
    int BackdropY;
    int StripNum;
    uint8 * StripDat;

    eSPECTRUM_OPERATE_CMD cmd;
}RKGUI_SPECTRUM_ARG;
```

名称	RKGUI_CHAIN_ARG
display	隐藏标志，0 隐藏，1 显示。
x	显示位置

y	显示位置
level	图层，0 底层，1 顶层（即状态栏）。
Backdrop	背景图片 SID。
BackdropX	背景 x 偏移
BackdropY	背景 y 偏移
StripNum	频谱条目数
StripDat	高度数据
cmd	操作命令选择： SPECTRUM_SET_CONTENT：修改高度

10.2.2 示例代码

```
HGC hGc;
RKGUI_SPECTRUM_ARG pstSpectrumContent;
uint8 buf[12]= {35,35,35,35,35,35,35,35,35,35,35,35}; // 12 根频谱高度（35 像素）

pstSpectrumArg.x= 23;           // 显示位置 x,y(23,29)
pstSpectrumArg.y= 29;
pstSpectrumArg.level= 0;        // 显示图层
pstSpectrumArg.display= 1;      // 创建即显示
pstSpectrumArg.StripNum= 12;     // 频谱总数
pstSpectrumArg.StripStyle= IMG_ID_MUSIC_SPECTRUM20; // 单根频谱图片编号
pstSpectrumArg.Backdrop= IMG_ID_MUSIC_BACKGROUND; // 背景图片编号
pstSpectrumArg.BackdropX= 23;   // 背景偏移
pstSpectrumArg.BackdropY= 29;
pstSpectrumArg.StripDat= buf;    // 数据
hGc= GUI_Task_CreateWidget(GUI_CLASS_SPECTRUM, &pstSpectrumArg);
```

10.3 内容变更

函数原型	rk_err_t GuiTask_OperWidget(HGC pGc, eRKGUI_OPERATE_CMD cmd, void *arg, uint32 mode)
pGc	控件句柄
cmd	OPERATE_SET_CONTENT。（替换内容时）
arg	RKGUI_SPECTRUM_ARG 具体意义与命令参数有关。

mode	SYNC_MODE: 同步模式 ASYNC_MODE: 异步模式
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

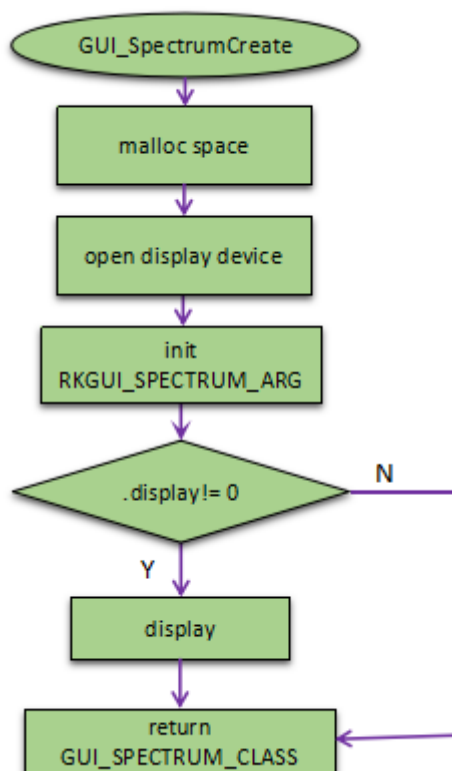
10.3.1 变更高度

```

RKGUI_SPECTRUM_ARG pstSpectrumContent;
pstSpectrumContent.cmd= SPECTRUM_SET_CONTENT;
pstSpectrumContent.StripDat= buf2;
GuiTask_OperWidget(hGc,      OPERATE_SET_CONTENT,      &pstSpectrumContent,
SYNC_MODE);

```

10.4 SPECTRUM 创建流程



PROGRESSBAR 控件的创建过程与其他控件基本一致。值得一提的是：为了加快处理速度，减少访问固件资源次数。SPECTRUM 在创建时，将会通过 rkos_memory_malloc 分配内存空间，并将整个图片资源读入内存，然后显示。这部分内存控件，将一直保持，直到控件被删除为止。

10.5 实现说明

10.5.1 GUI_SPECTRUM_CLASS

```
typedef struct _GUI_SPECTRUM_CLASS
{
    GUI_CONTROL_CLASS pfControl;

    int StripStyle;
    int Backdrop;
    int BackdropX;
    int BackdropY;
    int StripNum;
    uint8 * StripDat;
    uint8 * StripBuf;
    int StripXSize;
    int StripYSize;
}GUI_SPECTRUM_CLASS;
```

名称	GUI_SPECTRUM_CLASS
pfControl	控件类，包含了控件的基本信息。
StripStyle	频谱条 SID 编号
Backdrop	背景 SID 编号
BackdropX	背景偏移
BackdropY	背景偏移
StripNum	频谱条数量
StripDat	频谱高度数据
StripBuf	频谱点阵图形缓冲
StripXSize	频谱条单根宽度
StripYSize	频谱条单根高度

10.5.2 GUI_SelectCreate

函数原型	GUI_CONTROL_CLASS * GUI_SpectrumCreate(void *arg)
arg	创建参数，RKGUI_SPECTRUM_ARG 类型
返回值	NULL：创建失败

	非 NULL：控件句柄
功能说明	根据创建参数，创建控件，并返回控件句柄。

10.5.3 GUI_SpectrumDelete

函数原型	rk_err_t GUI_SpectrumDelete(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	根据句柄回收控件资源。

10.5.4 GUI_SpectrumDisplay

函数原型	rk_err_t GUI_SpectrumDisplay(HGC pGc)
pGc	控件句柄
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	显示控件。

10.5.5 GUI_SpectrumContent

函数原型	rk_err_t GUI_SpectrumContent(HGC pGc, void * arg)
pGc	控件句柄
arg	带入参数
返回值	RK_SUCCESS：执行成功 RK_ERROR：执行错误
功能说明	修改频谱条高度。

11 GUI 新增控件

11.1 GUI 控件概述

RKOS GUI 中提供了一系列用于完成用户界面开发的控件，帮助完成图片、文本框、下拉式菜单等界面元素的开发。当系统提供的控件种类不足以满足应用需求时，可以通过 RKOS GUI 提供的 API 和数据类型进行控件的新增。本章将以 ICON 控件为例，介绍 RKOS GUI 新增控件的方法。

11.2 控件类型声明

RKOS GUI 中所有的控件都必须在\rk-iot\Gui\GUITask.h 的 eGc 中具有申明，eGc 的定义如下所示：

```
typedef enum
{
    GUI_CLASS_IMAGE = 0,
    GUI_CLASS_ICON,
    GUI_CLASS_SELECT,
    GUI_CLASS_TEXT,
    GUI_CLASS_CHAIN,
    GUI_CLASS_MSG_BOX,
    GUI_CLASS_TEXTBOX,
    GUI_CLASS_PROGRESSBAR,
    GUI_CLASS_SPECTRUM,
} eGc;
```

11.3 控件类与控件子类

控件类是 RKOS GUI 内部用于记录控件类型、位置、大小等基本消息的数据类型。遮叠管理器模块将根据这些基础信息进行判断处理。因此，对于所有的控件而言，都必须包含控件类。并在创建时必须正确填写：

11.3.1 GUI_CONTROL_CLASS

```
typedef struct _GUI_CONTROL_CLASS
{
    struct _GUI_CONTROL_CLASS * next;
    struct _GUI_CONTROL_CLASS * prev;
    struct _GUI_CONTROL_CLASS * updatea;
    eGc ClassID;

    HDC hDisplay;

    int x0;
    int y0;
    int xSize;
    int ySize;
    uint32 display;
    RK_LCD_ROTATE rotate;
    int level;

    int flag;
    LUCENCY_TYPE lucency;
    int translucence;
    void * focus;
}GUI CONTROL CLASS;
```

名称	GUI_CONTROL_CLASS
next	链表指针，用于遮叠管理。（由遮叠管理器填写）
prev	链表指针，用于遮叠管理。（由遮叠管理器填写）
updatea	刷新指针，用于遮叠管理。（由遮叠管理器填写）
ClassID	控件类型，详见 eGc
hDisplay	Display 设备句柄。
x0	显示位置。
y0	显示位置。
xSize	控件大小。
ySize	控件大小。
display	隐藏标志，0 隐藏，1 显示
rotate	翻转标志： ROTATE_NOMAL：正常显示 ROTATE_FLIP_LR：左右镜像 ROTATE_FLIP_UD：上下镜像 ROTATE_ROTATE：180° 镜像

level	图层
flag	遮叠标志： 1 控件存在交叠 0 控件不存在交叠
lucency	透明模式： LUCENCY：透明模式 OPACITY：非透明模式
translucence	透明标志，暂无意义
focus	用于记录焦点回调函数。（由焦点管理器填写）

11.3.2 示例代码

RKOS GUI 中所有的控件都是 GUI_CONTROL_CLASS 的子类。控件子类可以根据需要，在包含 GUI_CONTROL_CLASS 类型的前提下定义其他参数：

```
typedef struct _GUI_ICON_CLASS
{
    GUI_CONTROL_CLASS pfControl;
    PICTURE_INFO_STRUCT psPictureInfo;
}GUI_ICON_CLASS;
```

11.4 创建与删除接口

11.4.1 创建与删除接口

当 GUITask 接收到控件创建/删除命令时，将调用\rk-iot\Gui\GUIManager.c 中的 GUI_GcCreate 和 GUI_GcDelete 函数，当我们需要新增/删除一个控件时，需要在此增加一个函数入口。

```
_GUI_GUITASK_COMMON_
COMMON API GUI_CONTROL_CLASS * GUI_GcCreate(eGc eGcId, void* arg)
{
    GUI_CONTROL_CLASS * pGUI= NULL;

    switch ( eGcId )
    {
        case GUI_CLASS_IMAGE:
            pGUI= GUI_ImageCreate(arg);
            break;
```



```
case GUI_CLASS_ICON:
    pGUI= GUI_IconCreate(arg);
    break;

case GUI_CLASS_TEXT:
    pGUI= GUI_TextCreate(arg);
    break;

case GUI_CLASS_CHAIN:
    pGUI= GUI_ChainCreate(arg);
    break;

case GUI_CLASS_MSG_BOX:
    pGUI= GUI_MsgBoxCreate(arg);
    break;

case GUI_CLASS_SELECT:
    pGUI= GUI_SelectCreate(arg);
    break;

case GUI_CLASS_PROGRESSBAR:
    pGUI= GUI_ProgressBarCreate(arg);
    break;

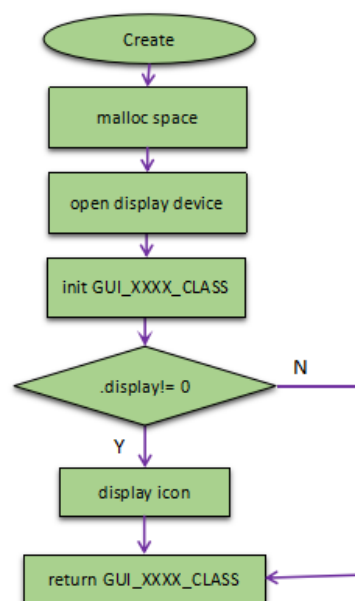
case GUI_CLASS_SPECTRUM:
    pGUI= GUI_SpectrumCreate(arg);
    break;

default:
    printf("nonsupport class id\n");
    return NULL;
}

GcRelevanceRegister(pGUI);
return pGUI;
}
```

11.4.2 初始化/反初始化规则

完整的控件创建流程，至少需要包含如下步骤：



当 GUITask 接收到控件创建请求时，将调用对应的初始化函数进行创建。控件创建的整个过程可以理解为是对控件子类的初始化，包括内存分配、开启 Display 设备、初始化控件子类、显示、返回句柄等步骤。控件创建完成后，GUI_GcCreate 函数将调用 GcRelevanceRegister 将控件句柄加入遮叠管理器链表，并将句柄传递给创建者。

反初始化步骤则是负责回收控件创建时所分配的资源、关闭 Display 设备等。值得注意的是，控件反初始化函数并不负责将控件从显示器上“消失”。若应用程序需要，可通过将控件隐藏，然后删除的方式实现。

11.5 控件操作接口

在实际应用中，我们常常需要改变控件的显示位置、内容等信息。RKOS GUI 中，当 GUITask 接收到这些操作请求时，将调用\rk-iot\Gui\GUIManager.c 中的 GUI_OperWidge 函数。此函数将根据控件类型及参数命令，调用控件的功能实现。

```
_GUI_GUITASK_COMMON_
COMMON API rk_err_t GUI_OperWidge(HGC hGc, eRKGUI_OPERATE_CMD cmd, void* arg)
{
    rk_err_t ret;
    GUI_CONTROL_CLASS *pGc= (GUI_CONTROL_CLASS *)hGc;

    if (pGc== NULL)
    {
        return RK_ERROR;
    }

    switch ( pGc->ClassID )
    {
        case GUI_CLASS_ICON:
        {
            switch (cmd)
            {
                case OPERATE_DISPLAY:
                    ret= GUI_IconDisplay(hGc);
                    break;

                case OPERATE_SET_DISPLAY:
                    ret= GUI_SetDisplay(hGc, (int)arg);
                    break;

                case OPERATE_SET_CONTENT:
                {
                    if( GUI_IconSetID(pGc, (uint32)arg) != RK_SUCCESS )
                    {
                        ret= RK_ERROR;
                    }
                }

                if(pGc->display)
                {
                    GcRelevanceOper(pGc);
                    ret= GcDisplay(pGc);
                }
            }
        }
    }
}
```

```
                break;
            }
        }
        break;
    }

    // ...

    return ret;
}
```

11.6 控件显示接口

当控件互相遮叠时，控件管理器将沿链表依次标记出所有需要刷新的控件。并沿着标记出的路径调用显示函数。原则上所有的控件都必须实现控件显示接口，并在\rk-iot\Gui\GUIManager.c中的增加对应的函数入口。

```
COMMON API rk_err_t GUI_DisplayWidge(HGC hGc)
{
    rk_err_t ret;
    GUI_CONTROL_CLASS *pGc= (GUI_CONTROL_CLASS *)hGc;

    if (pGc== NULL)
    {
        return RK_ERROR;
    }

    switch ( pGc->ClassID )
    {
        case GUI_CLASS_ICON:
            ret= GUI_IconDisplay(hGc);
            break;

        // ...

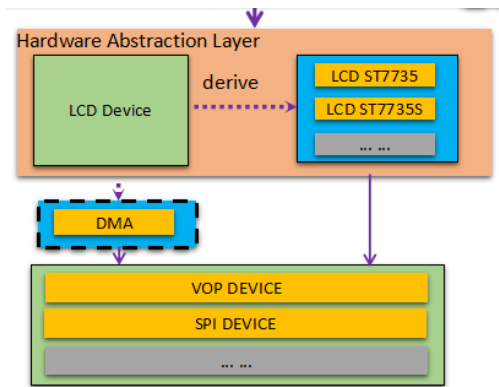
    }
    return ret;
}
```

12 硬件移植相关

12.1 移植概述

RKOS GUI 是一款能够适应任意尺寸 LCD 图形显示的图形用户接口。RKOS GUI 的硬件抽象层接口，能够屏蔽底层硬件细节。根据 RKOS GUI 框架设计的驱动程序，能够与系统无缝接合，从而获得上层所有平台无关代码的功能。

12.2 硬件抽象层架构



硬件抽象层为上层软件提供了虚拟的硬件平台，图中的 LCD Device 的本质是一个硬件无关的虚拟设备，其核心是一个用于描述硬件细节的接口类型。不同的 LCD 图形显示器可以通过此结构，注册功能函数；填写硬件参数；派生 LCD 设备类。上层软件将根据派生出的 LCD 设备，调用底层接口。

12.3 接口说明

从程序上看，在使用 RKOS GUI 相关的功能前，首先需要建立 GUI 设备链。以 LCD ST7735S 为例，依次需要建立 VOP 设备（总线控制器）、DMA 设备、LCD 设备、DISPLAY 设备、PWM 设备（背光控制）等。不同的 LCD 设备所需关联的设备存在一定差异。用户可以在 \Bsp\EVK_V2.0\BSP.c 的 CreateDeviceListDisplay 函数中找到详细的步骤。

12.3.1 LCD 设备参数

LCD 设备的建立采用 RKDev_Create 函数，完整的 GUI 设备链创建过程，可以参考 \Bsp\EVK_V2.0\BSP.c 中的 CreateDeviceListDisplay 函数。

函数原型	rk_err_t RKDev_Create(uint32 DevClassID, uint32 DevID, void* arg)
DevClassID	创建 LCD 设备时，固定为 DEV_CLASS_LCD
DevID	LCD 子设备号，始终为 0
arg	创建参数，LCD_DEV_ARG 类型
功能说明	创建 LCD 设备。

LCD_DEV_ARG 结构如下所示：

```
typedef struct _LCD_DEV_ARG
{
    HDC hBus;
    HDC hDma;
    uint16 dmaCh;
    RK_LCD_DRIVER *pLCDDriver;
}LCD_DEV_ARG;
```

名称	LCD_INFO_CLASS
hBus	总线设备句柄。
hDma	DMA 总线设备句柄，NULL 表示不支持 DMA。
dmaCh	DMA 通道号
pLCDDriver	LCD 接口类，详见后文。

12.3.2 硬件参数类

硬件参数类是接口类的子类，用于描述硬件设备长、宽等硬件参数细节。其定义如下：

```
typedef struct _LCD_INFO_CLASS
{
    char * name;

    uint16 width;
    uint16 height;

    uint32 pallet;
    uint16 BW;
}LCD_INFO_CLASS;
```

名称	LCD_INFO_CLASS
name	设备名称，仅调试时使用。
width	LCD 宽度，单位像素。
height	LCD 高度，单位像素。
pallet	调色模式，BCD 编码，分辨表示透明 RGC。例：RGB565 即 0x0565
BW	单色模式，0 彩色模式，1 单色模式

12.3.3 接口类

接口类用于注册上层软件所需使用的函数实体。需要指出的是：接口类型中并非所有的功能

函数都必须实现，由于硬件条件限制无法实现的功能只需填入 NULL。对于屏幕翻转等无法由硬件实现的功能，上层软件将通过算法途径实现，但是将有可能造成执行效率的降低。

```
typedef struct _RK_LCD_DRIVER
{
    LCD_INFO_CLASS LcdArg;

    void (*LcdInit) (HDC dev);

    rk_err_t (*LcdWakeUp) (void);
    rk_err_t (*LcdStandby) (void);

    rk_err_t (*LcdClear) (HDC dev, void * color);
    rk_err_t (*LcdSendData) (HDC dev, void * color);
    rk_size_t (*Lcd_Write) (HDC dev, void * color, uint32 size);
    rk_err_t (*LcdSetWindow) (HDC dev, int x0, int y0, int x1, int y1);
    rk_err_t (*LcdImage) (HDC dev, int x0, int y0, int x1, int y1, void * buf);

    rk_err_t (*LcdScreenUpdateDma) (HDC dev, HDC hDma, uint16 dmaCh,
    |int x0, int y0, int x1, int y1, void *pSrc, pFunc CallBack);

    rk_err_t (*LcdSetEdian) (HDC dev, uint32 edian);
    rk_err_t (*LcdSetRotate) (HDC dev, RK_LCD_ROTATE rotate);

    /* point the next driver*/
    void *next;
}RK_LCD_DRIVER;
```

12.3.4 LcdInit

函数原型	void LcdInit(HDC dev)
dev	设备句柄，无关变量，仅内部使用。
功能说明	初始化 LCD。

12.3.5 LcdWakeUp

函数原型	rk_err_t LcdWakeUp(void);
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	唤醒 LCD。

12.3.6 LcdStandby

函数原型	rk_err_t LcdStandby(void);
------	----------------------------

返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	休眠 LCD。

12.3.7 LcdClear

函数原型	rk_err_t LcdClear(HDC dev, void * color);
dev	设备句柄, 无关变量, 仅内部使用。
color	清屏颜色
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	将 LCD 显示器整屏填充为特定的颜色。

12.3.8 LcdSendData

函数原型	rk_err_t LcdSendData(HDC dev, void * color);
dev	设备句柄, 无关变量, 仅内部使用。
color	颜色
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	向 LCD 显示器写入一个像素。

12.3.9 Lcd_Write

函数原型	rk_size_t Lcd_Write(HDC dev, void * color, uint32 size);
dev	设备句柄, 无关变量, 仅内部使用。
color	数据指针
size	字节数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误

功能说明	将 LCD 显示器连续写入 size 个字节。起始位置由 LcdSetWindow 指定。
------	---

12.3.10 LcdSetWindow

函数原型	rk_size_t LcdSetWindow(HDC dev, int x0, int y0, int x1, int y1);
dev	设备句柄，无关变量，仅内部使用。
x0	x 轴起始坐标
y0	y 轴起始坐标
x1	x 轴终点坐标
y1	y 轴终点坐标
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	设置数据显示位置。

12.3.11 LcdImage

函数原型	rk_size_t LcdImage(HDC dev, int x0, int y0, int x1, int y1, void *pSrc);
dev	设备句柄，无关变量，仅内部使用。
x0	x 轴起始坐标
y0	y 轴起始坐标
x1	x 轴终点坐标
y1	y 轴终点坐标
pSrc	填充数据
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	将数据填充至特定的区域内。

12.3.12 LcdScreenUpdateDma

函数原型	rk_size_t LcdImage(HDC dev, HDC hDma, uint16 dmaCh, int x0, int y0, int
------	---

	x1 ,int y1 ,void *pSrc, pFunc CallBack);
dev	设备句柄，无关变量，仅内部使用。
hDma	DMA 设备句柄
dmaCh	DMA 通道
x0	x 轴起始坐标
y0	y 轴起始坐标
x1	x 轴像素个数
y1	y 轴像素个数
pSrc	填充数据
CallBack	传输完成回调函数
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	通过 DMA，向指定位置填充数据。传输完成后，将回调 CallBack。

12.3.13 LcdSetEdian

函数原型	rk_size_t LcdSetEdian(HDC dev,uint32 edian);
dev	设备句柄，无关变量，仅内部使用。
edian	大小端模式：0 : little-edian 1: big-edian
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	修改 LCD 显示设备的大小端模式

12.3.14 LcdSetRotate

函数原型	rk_size_t LcdSetRotate(HDC dev,RK_LCD_ROTATE rotate);
dev	设备句柄，无关变量，仅内部使用。
rotate	翻转模式： ROTATE_NOMAL 正常显示 ROTATE_FLIP_LR 左右镜像

	ROTATE_FLIP_UD 上下镜像 ROTATE_ROTATE 180° 镜像
返回值	RK_SUCCESS: 执行成功 RK_ERROR: 执行错误
功能说明	控制整个 LCD 屏幕翻转

12.4 步骤简述

将 RKOS GUI 移植到不同的 LCD 显示器中。首先，需要根据 RKOS 驱动架构编写驱动，完成 LCD 显示器相关的驱动程序的编写。然后，填写接口与硬件参数类，并在头文件中声明。最后在 \Bsp\EVK_V2.0\BSP.c 中包含对应的头文件，修改 \Bsp\EVK_V2.0\BSP.c 在开机时自动创建 GUI 设备链。创建 GUI 设备链的过程，可参见 \Bsp\EVK_V2.0\BSP.c 中的 CreateDeviceListDisplay 函数，在此不再赘述。若想了解更为完整的信息，可参见 \Gui\ST7735SDriver.c。

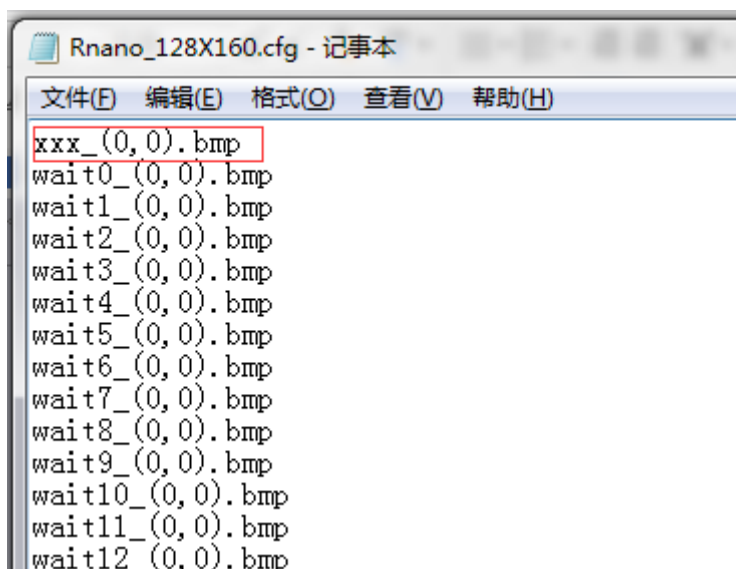
13 GUI 资源更新流程

13.1 工具简介

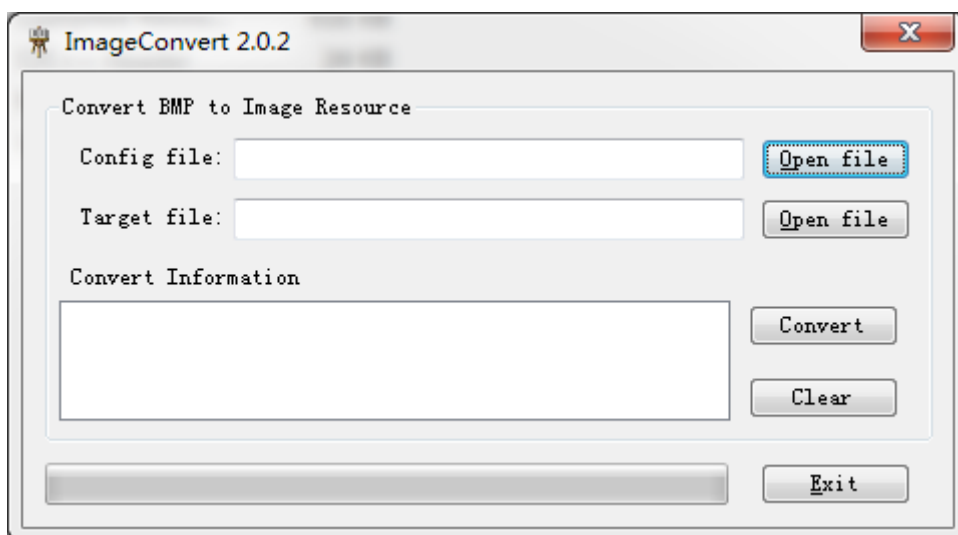
为方便用户定制 UI，RKOS 在 \rk-iot\Bsp\EVK_V2.0\Resource 路径下提供了一组工具软件，用于定制或替换程序中所使用到的图片或文字资源。

13.2 图片资源更新流程

1. 编辑图片，另存为 24 位位图格式。并以 xxx_(0,0).bmp 的方式命名。
2. 将文件保存至 \rk-iot\Bsp\EVK_V2.0\Resource\Picture 路径下。
3. 以文本编辑模式打开 \rk-iot\Bsp\EVK_V2.0\Resource\PictureRnano_128X160.cfg 文件，并插入待添加的文件名。



4. 打开\rk-iot\Bsp\EVK_V2.0\Resource\ImageConvert.exe。



5. Config File 配置文件路径: \rk-iot\Bsp\EVK_V2.0\Resource\PictureRnano_128X160.cfg

6. Target File 目标文件路径: \rk-iot\Bsp\EVK_V2.0\Resource\RkNanoImage.uis

7. 点击 Convert 生成固件资源

8. 软件将重新生成 RkNanoImage.uis 和 ImageResource.h 文件。其中 RkNanoImage.uis 存放了图片的点阵图形。ImageResource.h 中保存了图片的 SID 编号。

9. SPI 版: 将生成的 RkNanoImage.uis 文件存入文件系统根目录。

10. 修改代码、编译、下载。

13.3 菜单资源更新流程

1. 开启..\rk-iot\Bsp\EVK_V2.0\Resource\GenMeu\RKnano_COLOR.xls

/@ MenuResourceID @/	/@ Chinese S @/	/@ Chinese T @/
\$[X]MUSIC	\$[X]音乐播放	\$[X]音樂播放
\$[X]DLNA	\$[X]DLNA播放	\$[X]DLNA播放
\$[X]MAINMENU_BLUETOOTH	\$[X]蓝牙播放	\$[X]藍牙播放
\$[X]AIRPLAY	\$[X]AIRPLAY	\$[X]AIRPLAY
\$[X]5_1_CHANNEL	\$[X]5.1 声道	\$[X]5.1 声道
\$[X]RADIO	\$[X]收音机	\$[X]收音機
\$[X]RECORD	\$[X]录 音	\$[X]錄 音
\$[X]TEXT	\$[X]电子书	\$[X]電子書
\$[X]EXPLORER	\$[X]资源管理器	\$[X]資源管理器
\$[X]MAIN_SETTINGS	\$[X]设 置	\$[X]設 置
\$[X]VIDEO	\$[X]视频播放	\$[X]視頻播放
\$[X]PHOTO	\$[X]图片浏览	\$[X]圖片瀏覽

2. 该表中，第 1 列代表生成的 SID 的名称，从第 2 列开始的每一列对应一种语言文字。

3. 单元格内表示的是 SID 所对应的文字内容。分为标识符和内容两部分。

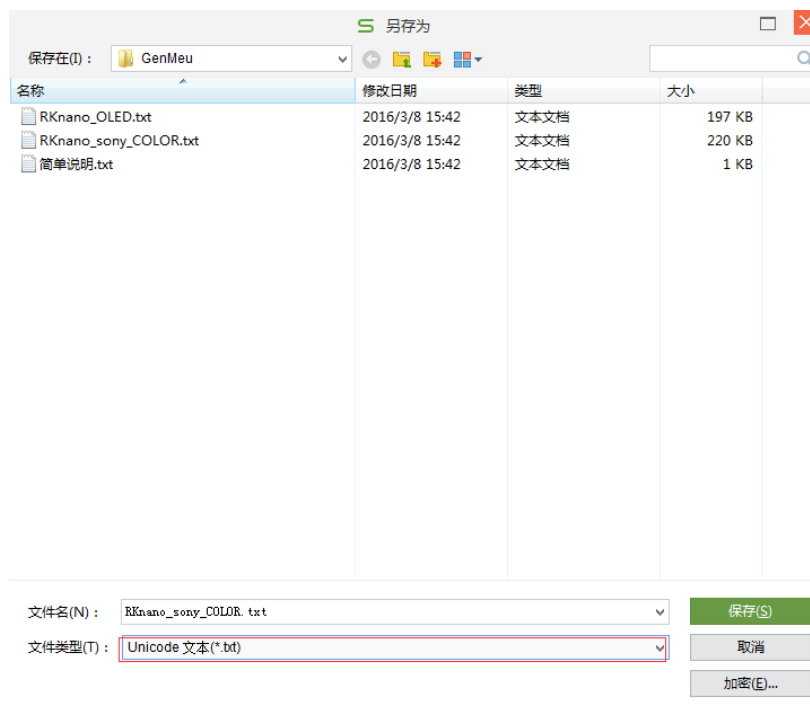
\$[X]: 字符常量。

#[1]: 一级标题，含有子菜单。

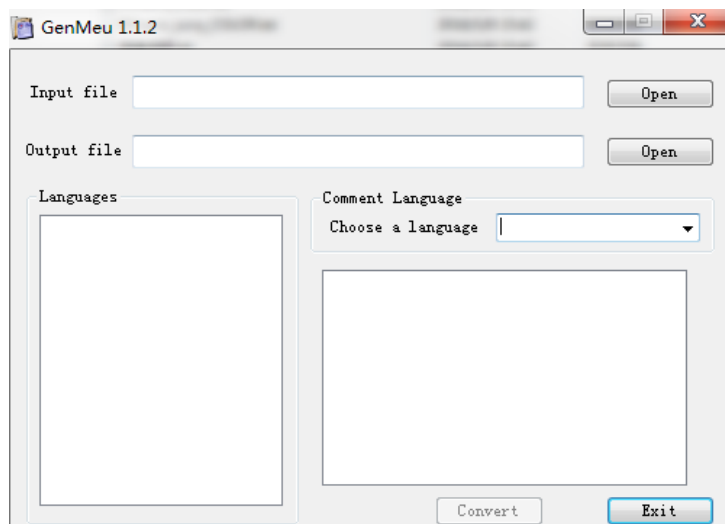
##[1]: 二级标题，含有子菜单。

###[0]: 三级标题，不含子菜单。

4. 将编辑好的 excel 表，另存为 unicode 文本。



5. 打开工具\rk-iot\Bsp\EVK_V2.0\ResourceGenMeu1.1.2.exe



6. Input File 输入文件路径: \rk-iot\Bsp\EVK_V2.0\Resource\RKnano_COLOR.txt
7. Output File 输出文件路径: \rk-iot\Bsp\EVK_V2.0\Resource\menu.meu
8. Choose a language: 选择注释语言
9. 点击 Convert
10. 软件将重新生成 menu.meu 和 MenuResourceID.h 文件。其中 MenuResourceID.h 中保存了菜单资源的 SID 编号。
11. SPI 版: 将 menu.meu 重命名为 menu.res 并存入文件系统根目录。
12. 修改代码、编译、下载。